

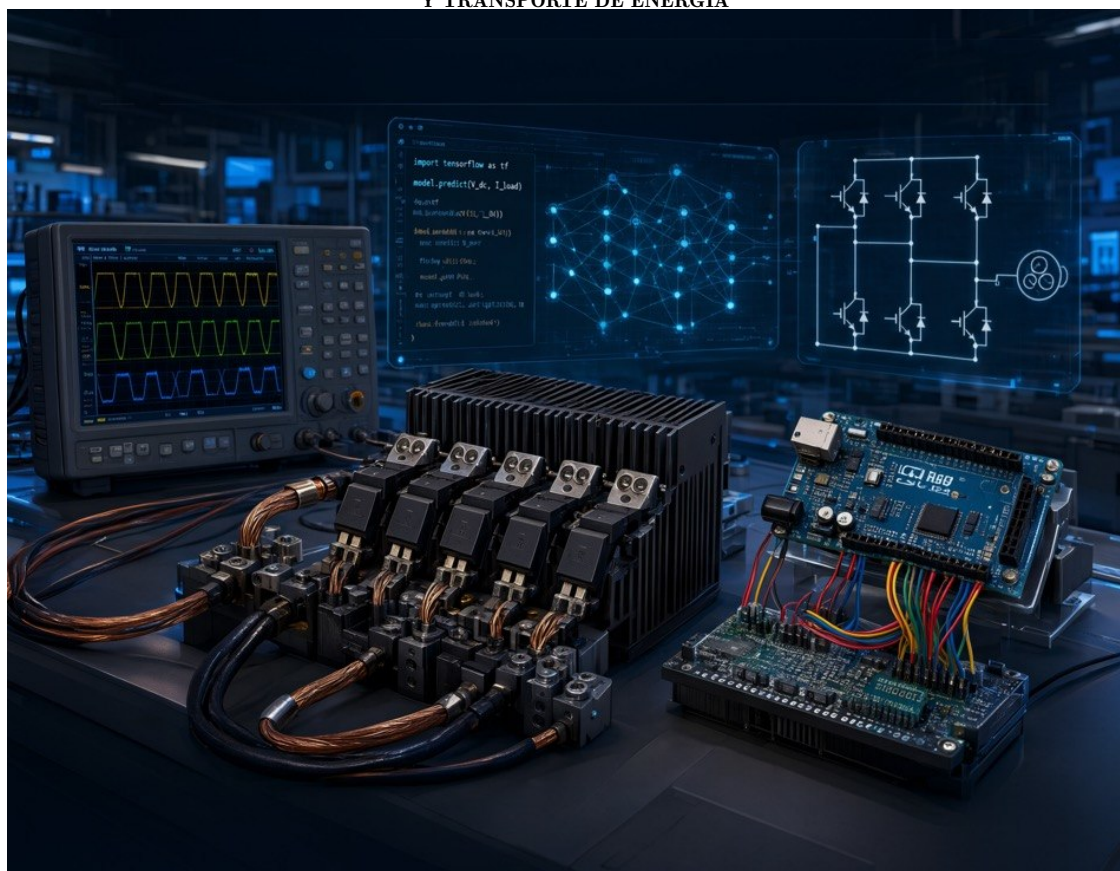
ANÁLISIS Y CONTROL DE INVERSORES DE POTENCIA

Integración de Técnicas de Modulación, Conmutación Digital y Etapas de Potencia

POR

ALEXANDER BUENO MONTILLA

PROFESOR TITULAR
DEPARTAMENTO DE CONVERSIÓN
Y TRANSPORTE DE ENERGÍA



UNIVERSIDAD SIMÓN BOLÍVAR
Caracas, Venezuela

Agosto 2026

Esta obra está bajo una licencia Creative Commons “Atribución-NoComercial-CompartirIgual 4.0 Internacional”.



Analisis y Control de Inversores de Potencia

Integracion de Tecnicas de Modulacion, Conmutacion Digital y Etapas de
Potencia

Prof. Alexander Bueno

Agosto 2026

Resumen

El presente documento integra cuatro trabajos técnicos relacionados con el diseño, análisis e implementación de sistemas de inversores de potencia controlados mediante la plataforma Arduino Uno. El contenido abarca desde el análisis detallado de etapas de potencia de medio puente con driver bootstrap discreto, hasta la implementación de estrategias de modulación PWM para inversores monofásicos y trifásicos, pasando por un estudio comparativo de métodos de conmutación digital de salidas.

El primer bloque analiza una etapa de potencia de medio puente basada en MOSFETs de canal N, accionada mediante un circuito de bootstrap discreto y señales TTL de hasta 10 kHz. Se estudian los modos de operación, los tiempos muertos generados por redes RC, el ciclo de trabajo máximo alcanzable y las pérdidas de potencia. El segundo bloque presenta tres metodologías para la conmutación de salidas digitales en Arduino Uno, ordenadas de mayor a menor nivel de abstracción: funciones `digitalWrite`, manipulación directa de registros y asignación atómica por máscara completa. El tercer bloque describe las técnicas de modulación aplicables a inversores monofásicos tipo puente H, incluyendo SPWM, THIPWM, MPWM y modulación por onda cuadrada, así como su implementación mediante el Timer1 del ATmega328P. El cuarto bloque expande el análisis a inversores trifásicos, abordando estrategias avanzadas como SVPWM, DPWM y modulación generalizada de vectores espaciales.

Se incluyen consideraciones de hardware, dimensionamiento de componentes, generación de tiempos muertos y un análisis comparativo que facilita la selección de la técnica más adecuada según la aplicación. El documento concluye con anexos que contienen los códigos de ejecución para Arduino y Python.

Índice general

1. Introduccion General	3
2. Etapa de Potencia de Medio Puente con Driver Bootstrap Discreto	5
2.1. Topologia y Principio de Funcionamiento	5
2.2. Modos de Operacion	6
2.2.1. Estado 1: $D_i = \text{ALTO (5 V)}$ — Salida en BAJO (0 V)	6
2.2.2. Estado 2: $D_i = \text{BAJO (0 V)}$ — Salida en ALTO (12 V)	6
2.3. Calculo de Tiempos Muertos	7
2.4. Ciclo de Trabajo Maximo	7
2.5. Analisis de Perdidas de Potencia	7
2.6. Recomendaciones de Diseno	8
2.7. Tabla de Componentes	8
3. Metodos de Control de Salidas Digitales en Arduino Uno	10
3.1. Arquitectura del Puerto B del ATmega328P	10
3.2. Comparacion de Metodos de Conmutacion	11
3.2.1. Metodo 1: Funciones de Alto Nivel (<code>digitalWrite</code>)	11
3.2.2. Metodo 2: Manipulacion Directa de Registros	11
3.2.3. Metodo 3: Asignacion Atomica por Mascara	11
3.3. Comparativa de Rendimiento	12
3.4. Recomendacion para Electronica de Potencia	12
4. Modulaciones para Inversor Monofasico Tipo Puente H	13
4.1. Principios de Modulacion PWM	13
4.1.1. Indices de Modulacion	13
4.2. Topologia del Inversor Puente H	14
4.3. Tecnicas de Modulacion para Puente H	14
4.3.1. Modulacion Senoidal (SPWM)	14
4.3.2. Modulacion con Inyeccion de Tercer Armonico (THIPWM)	15
4.3.3. Modulacion por Funcion Hiperbolica (MPWM)	16
4.3.4. Modulacion por Onda Cuadrada	17
4.4. Implementacion con el Timer1 del ATmega328P	18
5. Modulaciones para Inversor Trifasico	19
5.1. Topologia del Inversor Trifasico	19
5.2. Estrategias de Modulacion para Trifasico	19
5.2.1. Modulacion Vectorial (SVPWM)	19
5.2.2. Modulacion Discontinua (DPWM)	20
5.2.3. Modulacion Generalizada de Vectores Espaciales	20

5.2.4. Comparativa de Tecnicas Avanzadas	21
6. Comparativa General y Criterios de Selecccion	22
6.1. Resumen de Tecnicas de Modulacion	22
6.2. Criterios de Selecccion	22
6.3. Recomendaciones para la Implementacion	23
7. Conclusiones Generales	24
A. Anexos: Codigos de Ejecucion	26
A.1. Anexo A: Codigo Arduino para Generacion de SPWM con Timer1	26
A.2. Anexo B: Codigo Python para Generacion de Tablas de Modulacion	27
A.3. Anexo C: Codigo Arduino para SVPWM Generalizado	29
B. Anexos: Códigos de Práctica Monofásico	32
B.1. Inversor Monofásico	32
B.2. Inversor Monofásico Modulaci3n	40
C. Anexos: Codigos de Practica Trifasico	52
C.1. Inversor Trifasico	52
C.2. Inversor Trifasico Modulaci3n	60
Bibliografia	75

Capítulo 1

Introduccion General

Los convertidores de corriente directa a corriente alterna (DC-AC), comunmente denominados inversores, constituyen uno de los bloques fundamentales en los sistemas de electronica de potencia moderna. Su aplicacion abarca desde fuentes de alimentacion in-interrumpida (UPS) y sistemas de generacion distribuida, hasta el control de motores de induccion en aplicaciones industriales y de traccion electrica. La topologia mas extendida para aplicaciones monofasicas de baja y media potencia es el inversor tipo puente H, mientras que para sistemas trifasicos se emplea la configuracion de tres ramas de medio puente, cada una con dos interruptores de potencia (MOSFET o IGBT) que conmutan para generar tensiones alternas a partir de una fuente de continua en topologia VSI (*Voltage Source Inverter*).

El control de estos inversores requiere la generacion de senales de conmutacion precisas, con tiempos muertos adecuados para evitar la conduccion cruzada, y la implementacion de estrategias de modulacion que determinen la calidad de la tension de salida y la eficiencia del sistema. La plataforma Arduino Uno, basada en el microcontrolador ATmega328P a 16 MHz, ofrece una solucion accesible y versatil para implementar algoritmos de control en tiempo real.

El presente documento integra cuatro trabajos tecnicos que abordan de manera secuencial y didactica los aspectos fundamentales del diseno, analisis e implementacion de inversores de potencia controlados por Arduino:

1. **Analisis de una etapa de potencia de medio puente** con driver bootstrap discreto, que constituye el bloque basico de cualquier inversor. Se estudian los modos de operacion, el dimensionamiento de componentes y el calculo de tiempos muertos mediante redes RC.
2. **Metodos de conmutacion de salidas digitales** en el Arduino Uno, analizando el compromiso entre portabilidad, velocidad y seguridad operativa. Se presentan tres tecnicas ordenadas de mayor a menor nivel de abstraccion, destacando la asignacion atomica por mascara como la mas adecuada para aplicaciones de electronica de potencia.
3. **Tecnicas de modulacion para inversores monofasicos tipo puente H**, incluyendo la Modulacion por Ancho de Pulso Senoidal (SPWM), la Modulacion con Inyeccion de Tercer Armonico (THIPWM), la Modulacion por Funcion de Transferencia Hiperbolica (MPWM) y la modulacion por Onda Cuadrada. Se analizan los parametros de diseno, el contenido armonico y el rendimiento de cada tecnica.

4. **Estrategias de modulación para inversores trifásicos**, expandiendo el análisis a técnicas avanzadas como la Modulación por Vectores Espaciales (SVPWM), la Modulación Discontinua (DPWM) y la modulación generalizada de vectores espaciales. Se incluye un estudio detallado del parámetro δ que permite unificar todas las estrategias en un único algoritmo.

La organización del documento sigue un orden lógico que va desde el análisis de la etapa de potencia individual, pasando por los métodos de control digital, hasta la implementación de estrategias de modulación completas para sistemas monofásicos y trifásicos. Se complementa con consideraciones de hardware, dimensionamiento de componentes y generación de tiempos muertos, así como un análisis comparativo que facilita la selección de la técnica más adecuada según la aplicación.

Los anexos incluyen los códigos de ejecución para Arduino (lenguaje C++) y Python, que permiten la generación de tablas de modulación y la configuración del Timer1 del ATmega328P. Todo el contenido está orientado a proporcionar una base sólida tanto para fines educativos como para el desarrollo de prototipos funcionales en el ámbito de la electrónica de potencia.

Capítulo 2

Etapa de Potencia de Medio Puente con Driver Bootstrap Discreto

2.1. Topologia y Principio de Funcionamiento

El circuito de medio puente constituye la base de cualquier inversor de potencia. La topologia emplea dos MOSFETs de canal N: uno para el lado alto (*high-side*) y otro para el lado bajo (*low-side*), alimentados desde una fuente de tension continua. Dado que los MOSFETs de canal N requieren una tension de puerta superior a la tension de fuente para conducir, el lado alto necesita un suministro flotante, que se obtiene mediante un condensador de *bootstrap*.

El diseno propuesto destaca por su simplicidad, al emplear componentes discretos (transistor BJT, diodos y redes RC) para generar los retardos necesarios que eviten la conduccion cruzada. El circuito se controla mediante una senal digital TTL de hasta 10 kHz, que gobierna el transistor de accionamiento Q_{DRV} y, a traves de el, los estados de los MOSFETs.

La Figura ?? muestra la topologia completa del medio puente con driver bootstrap discreto.

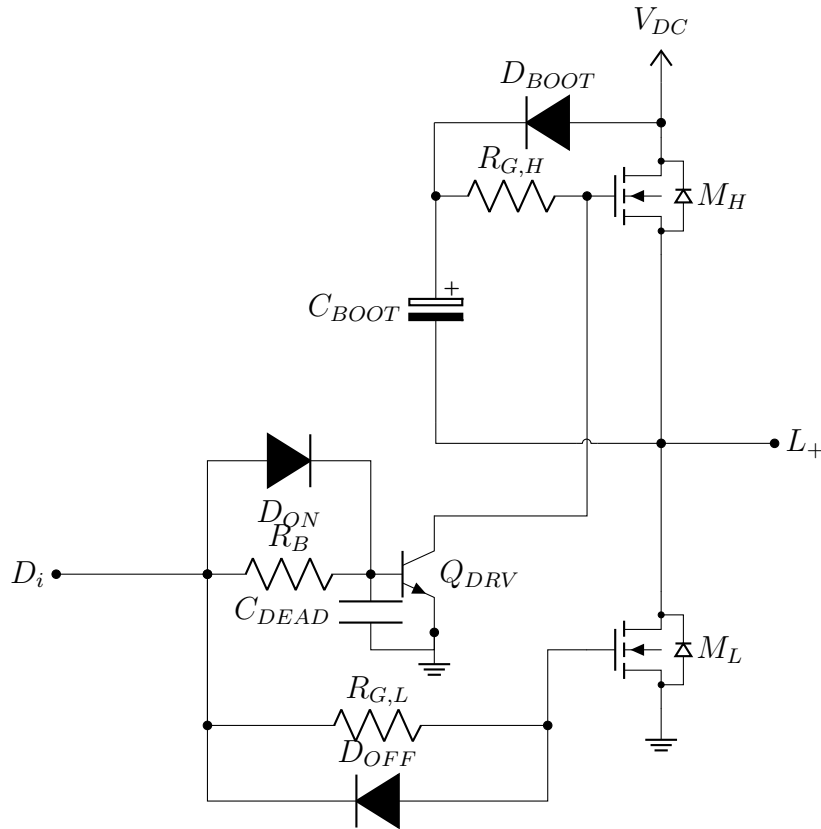


Figura 2.1: Rama de medio puente con driver bootstrap discreto y nomenclatura generica.

2.2. Modos de Operacion

El funcionamiento del circuito se describe en dos estados principales:

2.2.1. Estado 1: $D_i = \text{ALTO}$ (5 V) — Salida en BAJO (0 V)

El transistor Q_{DRV} conduce rapidamente gracias al diodo D_{ON} , que polariza directamente su base. El colector de Q_{DRV} pone a masa la compuerta del MOSFET alto M_H , manteniendolo apagado. La senal D_i tambien polariza la compuerta del MOSFET bajo M_L a traves de $R_{G,L}$, haciendo que M_L conduzca y fije la salida a tierra. Durante este intervalo, el condensador de *bootstrap* C_{BOOT} se carga a traves de D_{BOOT} y $R_{G,H}$ desde la alimentacion V_{DC} , alcanzando aproximadamente 11,3 V (descontando la caida del diodo).

2.2.2. Estado 2: $D_i = \text{BAJO}$ (0 V) — Salida en ALTO (12 V)

Cuando D_i pasa a nivel bajo, M_L se apaga rapidamente gracias al diodo D_{OFF} , que descarga su compuerta. Simultaneamente, la base de Q_{DRV} pierde corriente, pero el condensador C_{DEAD} y la resistencia R_B introducen un retardo en el apagado de Q_{DRV} (aproximadamente 4 μs). Durante este retardo, ambos MOSFETs permanecen apagados (tiempo muerto). Una vez que Q_{DRV} se corta, la compuerta de M_H es elevada por el voltaje almacenado en C_{BOOT} a traves de $R_{G,H}$, encendiendo M_H y llevando la salida a 12 V. En este momento, el terminal inferior de C_{BOOT} esta a 12 V, por lo que su terminal superior

(compuerta de M_H) alcanza aproximadamente 23,3 V, garantizando un encendido firme de M_H .

2.3. Calculo de Tiempos Muertos

Los tiempos muertos se generan por las constantes de tiempo de las redes RC asociadas a las compuertas y a la base de Q_{DRV} . Para los valores estandar:

- $R_B = 2,2 \text{ k}\Omega$, $C_{DEAD} = 1 \text{ nF} \Rightarrow \tau_{off} = R_B C_{DEAD} = 2,2 \text{ }\mu\text{s}$.
- Tension de base de Q_{DRV} en conduccion: $\sim 4,3 \text{ V}$; tension de umbral de corte: $\sim 0,7 \text{ V}$.
- Tiempo de apagado de Q_{DRV} :

$$t_{off} = \tau_{off} \ln \left(\frac{4,3}{0,7} \right) \approx 4 \text{ }\mu\text{s}.$$

Este retardo, sumado al tiempo de carga de la compuerta de M_H a traves de $R_{G,H}$ ($220 \text{ }\Omega$), da un tiempo muerto total en la transicion D_i ALTO→BAJO de aproximadamente 4,07 μs . En la transicion inversa (BAJO→ALTO), el apagado de M_H es casi instantaneo (gracias a Q_{DRV}) y el encendido de M_L tarda unos 1,37 μs , resultando en un tiempo muerto de 1,27 μs . Ambos tiempos son suficientes para evitar la conduccion cruzada a 10 kHz.

2.4. Ciclo de Trabajo Maximo

El periodo de conmutacion es $T = 100 \text{ }\mu\text{s}$ (para 10 kHz). El ciclo de trabajo D se define como la fraccion del periodo en que M_H conduce. Para que el circuito opere de forma segura, es necesario que exista un tiempo minimo de apagado de M_H que permita la recarga del condensador de *bootstrap* y complete el tiempo muerto. El tiempo muerto critico es $t_{dead2} \approx 4 \text{ }\mu\text{s}$. Por tanto:

$$(1 - D)T \geq t_{dead2} \Rightarrow D \leq 1 - \frac{4}{100} = 0,96.$$

El ciclo de trabajo maximo teorico es del 96 %, aunque en la practica se recomienda no superar el 90 % para garantizar la recarga del *bootstrap* y minimizar perdidas.

2.5. Analisis de Perdidas de Potencia

Para una corriente de carga de 1 A, un ciclo de trabajo del 50 % y los valores de resistencia de compuerta indicados, se estiman las siguientes perdidas:

Tabla 2.1: Perdidas estimadas para el medio puente.

Tipo de Perdida	Valor Estimado
Conduccion	24 mW
Conmutacion	98 mW
Accionamiento de compuerta	8 mW
Recuperacion inversa de D_{BOOT}	60 mW

Las pérdidas totales rondan los 190 mW, lo que da una eficiencia aproximada del 98 % para una salida de 12 W.

2.6. Recomendaciones de Diseño

Las principales debilidades del diseño son:

- **Uso de diodos 1N4007:** Presentan una recuperación inversa lenta para 10 kHz; se recomienda reemplazarlos por UF4007 o 1N4148.
- **Resistencia de compuerta del lado bajo** ($R_{G,L} = 1,5 \text{ k}\Omega$): Es excesiva, provocando pérdidas de conmutación elevadas; reducirla a 220Ω mejorará la eficiencia.
- **Generación de tiempos muertos mediante RC:** Es sensible a tolerancias y temperatura; para aplicaciones críticas, es preferible usar un *driver* integrado de medio puente (ej. IR2101).

2.7. Tabla de Componentes

La Tabla ?? resume los componentes del circuito de medio puente.

Tabla 2.2: Componentes del circuito de medio puente y sus valores.

Referencia	Tipo/Descripcion	Valor	Funcion
M_H, M_L	MOSFET de canal N	IRFZ48N (o equivalente)	Conmutadores de potencia
Q_{DRV}	BJT NPN	2N2222 (o equivalente)	Inversor/desplazador de nivel
D_{BOOT}	Diodo de bootstrap	1N4007 (recomendado UF4007)	Carga de C_{BOOT}
D_{OFF}	Diodo de descarga rapida	1N4007 (recomendado UF4007)	Acelera apagado de M_L
D_{ON}	Diodo de encendido rapido	1N4007	Acelera encendido de Q_{DRV}
$R_{G,H}$	Resistencia de compuerta (alto)	220 Ω	Limita corriente de compuerta de M_H
$R_{G,L}$	Resistencia de compuerta (bajo)	1,5 k Ω	Limita corriente de compuerta de M_L
R_B	Resistencia de base	2,2 k Ω	Define tiempo muerto junto con C_{DEAD}
C_{BOOT}	Condensador de bootstrap	10 μ F	Fuente flotante para M_H
C_{DEAD}	Condensador de tiempo muerto	1 nF	Genera retardo de apagado de Q_{DRV}
V_{DC}	Alimentacion de potencia	12 V	Tension del bus
D_i	Entrada de control	TTL (0 V–5 V)	Senal PWM de 10 kHz
L_+	Salida de potencia	—	Nodo de conexion a la carga

Capítulo 3

Metodos de Control de Salidas Digitales en Arduino Uno

3.1. Arquitectura del Puerto B del ATmega328P

El ATmega328P gestiona los pines del Puerto B (D8 a D13) mediante tres registros:

- DDRB (Data Direction Register B): configura la direccion (entrada/salida).
- PORTB (Port B Data Register): escribe el nivel logico en los pines configurados como salida.
- PINB (Port B Input Pins Address): lee el estado de los pines configurados como entrada.

Los pines D9 (PB1), D10 (PB2) y D11 (PB3) son especialmente adecuados para gobernar las ramas del inversor, ya que pertenecen al mismo puerto y permiten manipulacion directa de registros, facilitando la actualizacion simultanea de las salidas. La Figura ?? muestra la arquitectura simplificada del mapeo entre el nucleo, los registros y los pines.

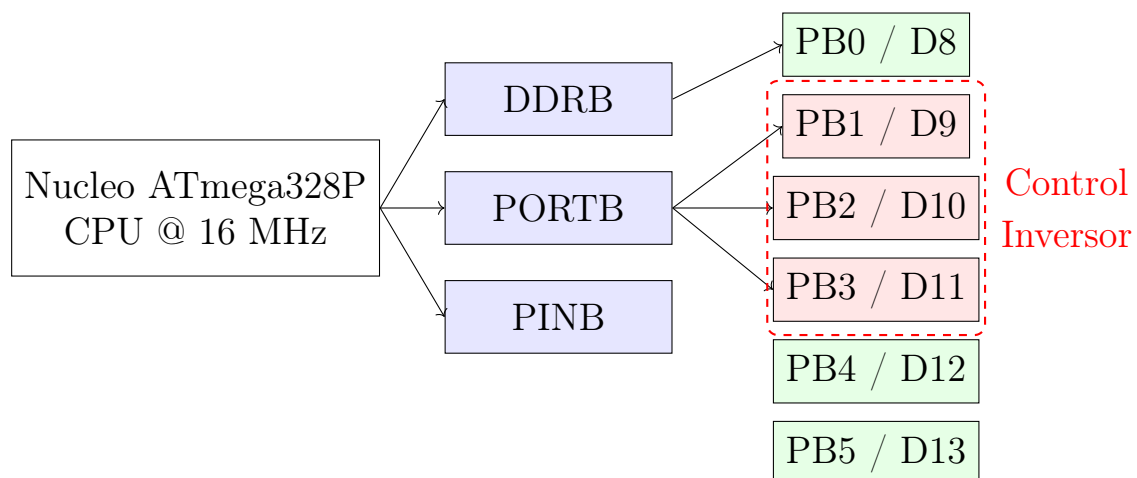


Figura 3.1: Arquitectura simplificada del mapeo entre el nucleo, los registros del Puerto B y los pines fisicos.

3.2. Comparacion de Metodos de Conmutacion

A continuacion se presentan tres metodos para actualizar las salidas digitales, ordenados de mayor a menor nivel de abstraccion.

3.2.1. Metodo 1: Funciones de Alto Nivel (digitalWrite)

```
1 void setPinsMethod1(bool d9High) {
2   if (d9High) {
3     digitalWrite(9, HIGH);
4     digitalWrite(10, LOW);
5   } else {
6     digitalWrite(9, LOW);
7     digitalWrite(10, HIGH);
8   }
9 }
```

Listing 3.1: Metodo 1: digitalWrite

Ventajas: Portabilidad, legibilidad, mantenibilidad.

Desventajas: Alta latencia (3.5–6 us por llamada), no atomicidad, jitter.

3.2.2. Metodo 2: Manipulacion Directa de Registros

```
1 void setPinsMethod2(bool d9High) {
2   if (d9High) {
3     PORTB |= (1 << PB1);    // D9 HIGH
4     PORTB &= ~(1 << PB2);   // D10 LOW
5   } else {
6     PORTB &= ~(1 << PB1);   // D9 LOW
7     PORTB |= (1 << PB2);    // D10 HIGH
8   }
9 }
```

Listing 3.2: Metodo 2: manipulacion de bits

Ventajas: Velocidad (250 ns), determinismo.

Desventajas: Riesgo de estados transitorios no seguros, dependencia de arquitectura.

3.2.3. Metodo 3: Asignacion Atomica por Mascara

```
1 void setOutputsAtomic(uint8_t d9_state, uint8_t d10_state) {
2   uint8_t mask = (1 << PB1) | (1 << PB2);
3   uint8_t desired = (d9_state ? (1 << PB1) : 0) |
4                     (d10_state ? (1 << PB2) : 0);
5   PORTB = (PORTB & ~mask) | desired;
6 }
```

Listing 3.3: Metodo 3: escritura atomica

Ventajas: Atomicidad, maxima velocidad (187 ns), seguridad en potencia.

Desventajas: Mayor complejidad sintactica, riesgo de corrupcion de bits adyacentes.

3.3. Comparativa de Rendimiento

La Tabla ?? resume los parametros de rendimiento de los tres metodos.

Tabla 3.1: Comparativa de metodos de control digital.

Parametro	Metodo 1	Metodo 2	Metodo 3
Tiempo de ejecucion	4–6 us	250 ns	187 ns
Atomicidad	No	No	Si
Portabilidad	Alta	Baja	Baja
Legibilidad	Alta	Media	Baja
Seguridad en puente H	Baja	Media	Alta
Determinismo temporal	Bajo	Alto	Alto
Uso de memoria de programa	Alto	Bajo	Bajo

La Figura ?? muestra la comparacion grafica de los tiempos de ejecucion.

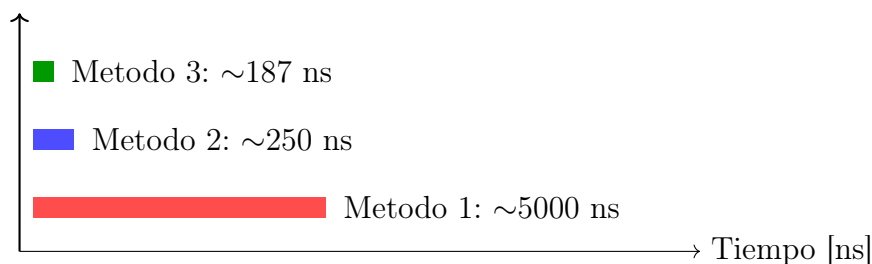


Figura 3.2: Comparacion de tiempos de ejecucion para la conmutacion de salidas.

3.4. Recomendacion para Electronica de Potencia

Para aplicaciones de electronica de potencia, el Metodo 3 es el mas seguro y eficiente. Se debe prestar especial atencion a la mascara para no alterar los pines PB0, PB4 y PB5. La funcion `setOutputsAtomic` encapsula de manera robusta y eficiente la escritura atomica, proporcionando una herramienta fiable para el control de salidas criticas.

Capítulo 4

Modulaciones para Inversor Monofasico Tipo Puente H

4.1. Principios de Modulacion PWM

La modulacion PWM consiste en comparar una senal de referencia (generalmente senoidal) con una portadora de alta frecuencia (diente de sierra o triangular). El resultado de la comparacion genera una senal binaria cuyo ciclo de trabajo varia segun la amplitud instantanea de la referencia.

4.1.1. Indices de Modulacion

- **Indice de modulacion de amplitud** (m_a): Relacion entre la amplitud pico de la referencia y la amplitud pico de la portadora.

$$m_a = \frac{\hat{V}_{ref}}{\hat{V}_{port}}$$

Para $m_a \leq 1$ se opera en la region lineal; si $m_a > 1$ se produce sobremodulacion.

- **Indice de modulacion de frecuencia** (m_f): Relacion entre la frecuencia de la portadora y la frecuencia de la referencia.

$$m_f = \frac{f_{port}}{f_{ref}}$$

Un m_f entero e impar favorece la cancelacion de armonicos en la tension de salida.

La Figura ?? ilustra el principio de la modulacion PWM.

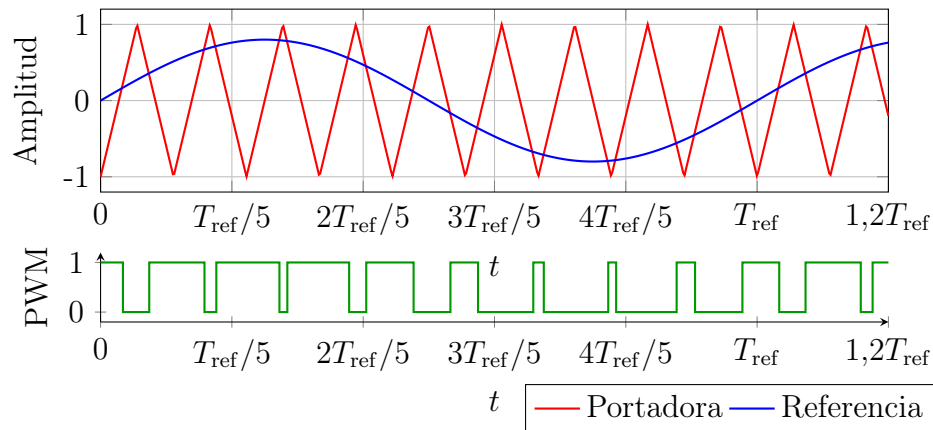


Figura 4.1: Principio de la modulacion PWM: referencia senoidal, portadora triangular y senal PWM resultante.

4.2. Topologia del Inversor Puente H

El inversor monofasico tipo puente H esta compuesto por cuatro interruptores de potencia organizados en dos ramas. La carga se conecta entre los puntos medios de ambas ramas. La Figura ?? muestra la topologia.

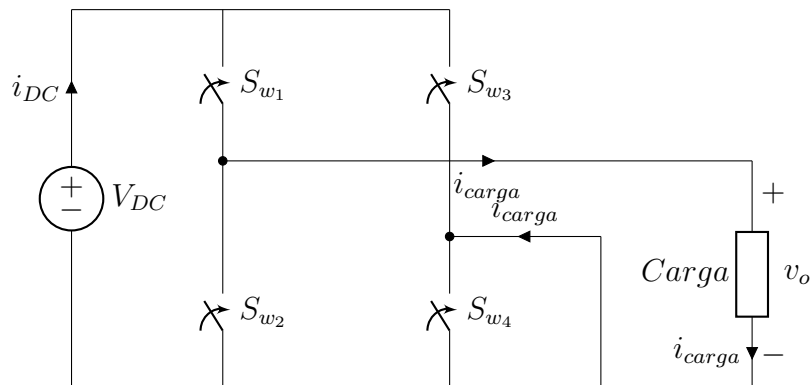


Figura 4.2: Topologia basica del inversor monofasico tipo puente H.

4.3. Tecnicas de Modulacion para Puente H

4.3.1. Modulacion Senoidal (SPWM)

La Modulacion por Ancho de Pulso Senoidal (SPWM) utiliza una referencia senoidal pura:

$$v_{ref}(t) = m_a \cdot \sin(\omega_{ref}t)$$

La comparacion con la portadora triangular determina los instantes de conmutacion. La Figura ?? muestra las senales caracteristicas.

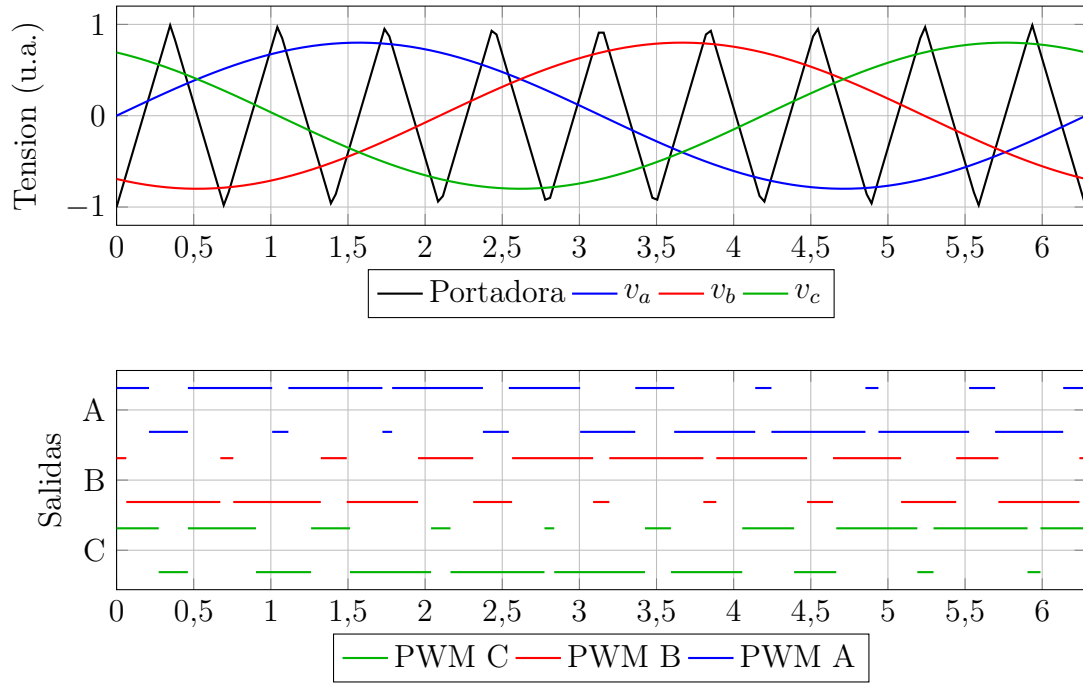


Figura 4.3: Modulación SPWM. Subgráfica superior: referencias y portadora. Subgráfica inferior: salidas PWM con separación vertical.

4.3.2. Modulación con Inyección de Tercer Armonico (THIPWM)

Se anade una componente de tercer armonico a la referencia senoidal:

$$v_{ref}(t) = \sin(\omega t) + a_3 \sin(3\omega t), \quad a_3 \approx 0,25$$

Esta tecnica permite m_a hasta 1.15, mejorando el uso del bus DC. La Figura ?? muestra las senales.

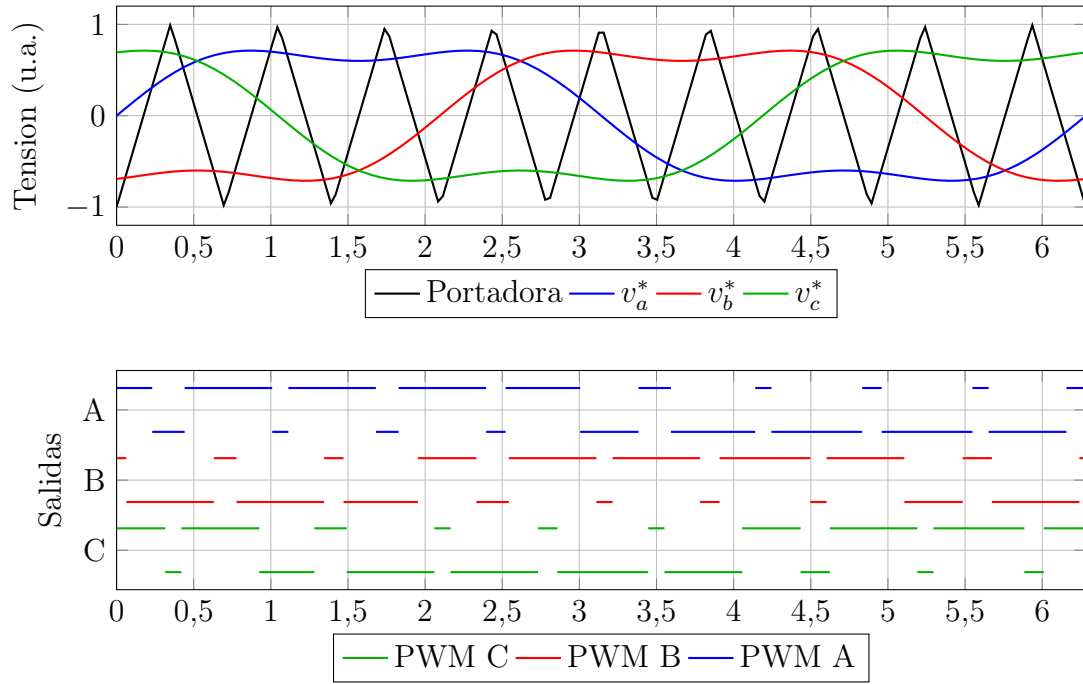


Figura 4.4: Modulacion THIPWM. Subgrafica superior: referencias con tercer armonico y portadora. Subgrafica inferior: salidas PWM con separacion vertical.

4.3.3. Modulacion por Funcion Hiperbolica (MPWM)

Emplea una funcion tangente hiperbolica:

$$v_{ref}(t) = \frac{\tanh(k \sin(\omega t))}{\max |\tanh(k \sin(\omega t))|}$$

La Figura ?? muestra las senales para $k = 2$.

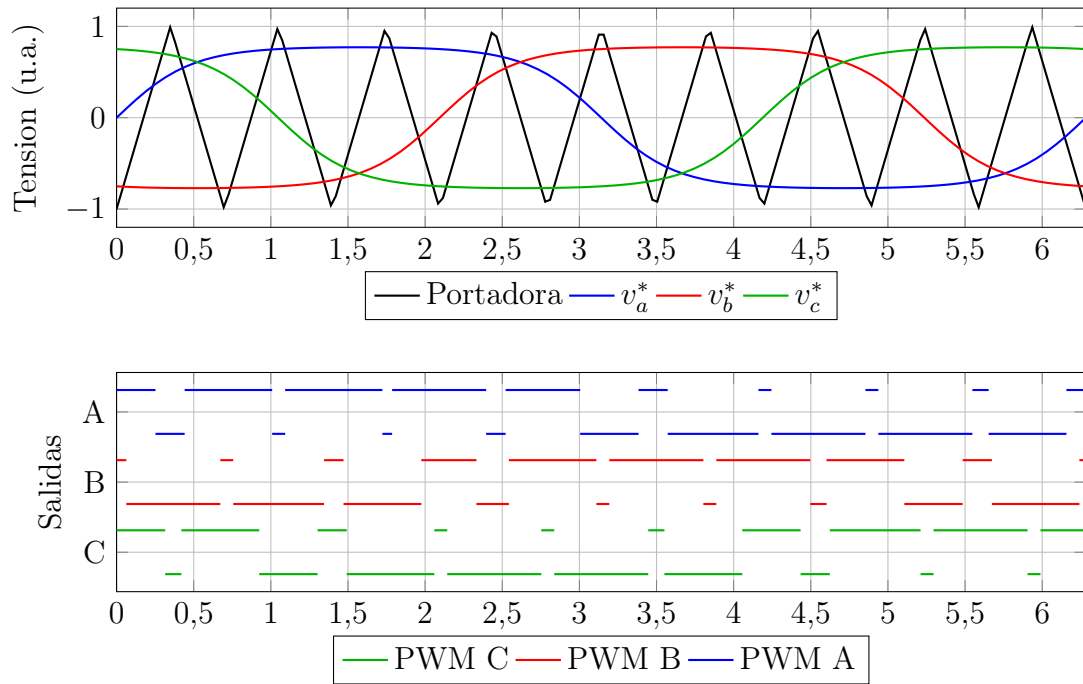


Figura 4.5: Modulación MPWM. Subgráfica superior: referencias hiperbolicas y portadora. Subgráfica inferior: salidas PWM con separacion vertical.

4.3.4. Modulación por Onda Cuadrada

Los interruptores conmutan a la frecuencia fundamental sin PWM. La tension de salida es:

$$v_o(t) = \frac{4V_{DC}}{\pi} \sum_{n=1,3,5,\dots} \frac{1}{n} \sin(n\omega t)$$

La THD teorica es del 48.3%. La Figura ?? muestra las senales.

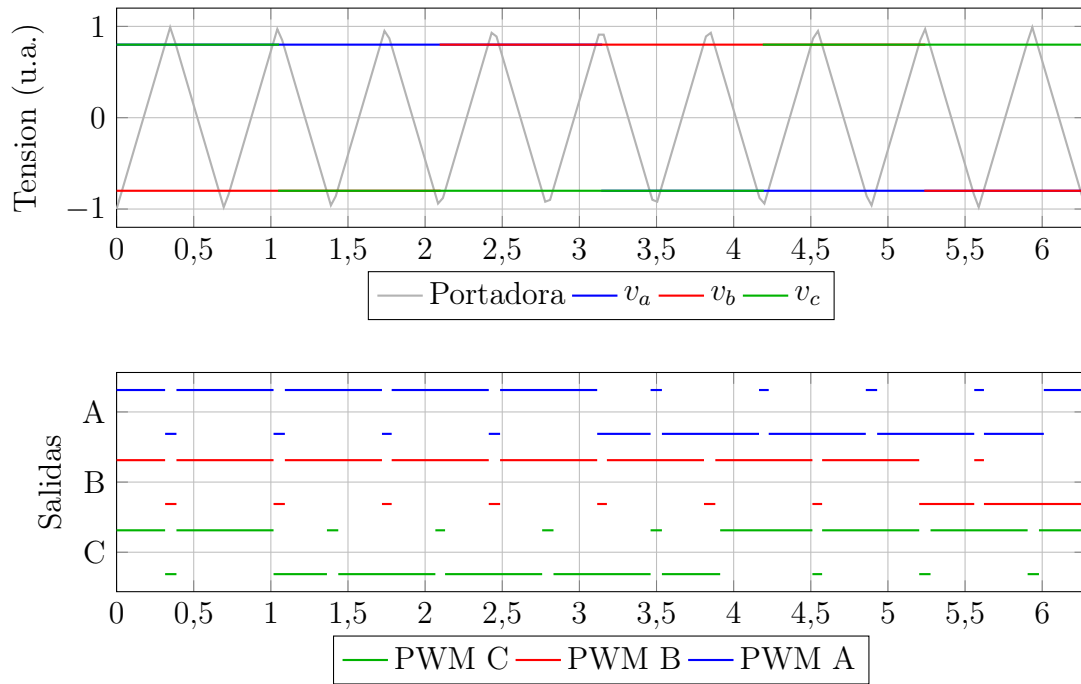


Figura 4.6: Modulación por onda cuadrada. Subgráfica superior: referencias cuadradas y portadora (gris). Subgráfica inferior: señales de conmutación con separación vertical.

4.4. Implementación con el Timer1 del ATmega328P

El Timer1 se configura en modo Fast PWM con TOP programable por ICR1. La frecuencia de la PWM se determina por:

$$f_{PWM} = \frac{f_{clk}}{N_{prescaler} \cdot (ICR1 + 1)}$$

Los registros OCR1A y OCR1B controlan los ciclos de trabajo de las ramas A y B. La Tabla ?? muestra una configuración típica.

Tabla 4.1: Ejemplo de configuración para inversor SPWM con Arduino Uno.

Parametro	Valor
Frecuencia de referencia (f_{ref})	60 Hz
Indice de modulación de amplitud (m_a)	0.80
Indice de modulación de frecuencia (m_f)	12
Frecuencia de portadora (f_c)	720 Hz
Tiempo muerto (t_{dead})	2.0 μ s
Preescalador seleccionado	8
Valor de ICR1	3332
Frecuencia real de conmutación	599.82 Hz
Ticks de tiempo muerto	4

Capítulo 5

Modulaciones para Inversor Trifasico

5.1. Topologia del Inversor Trifasico

El inversor trifasico esta compuesto por tres ramas de medio puente, cada una controlada por una senal PWM. La Figura ?? muestra la topologia completa.

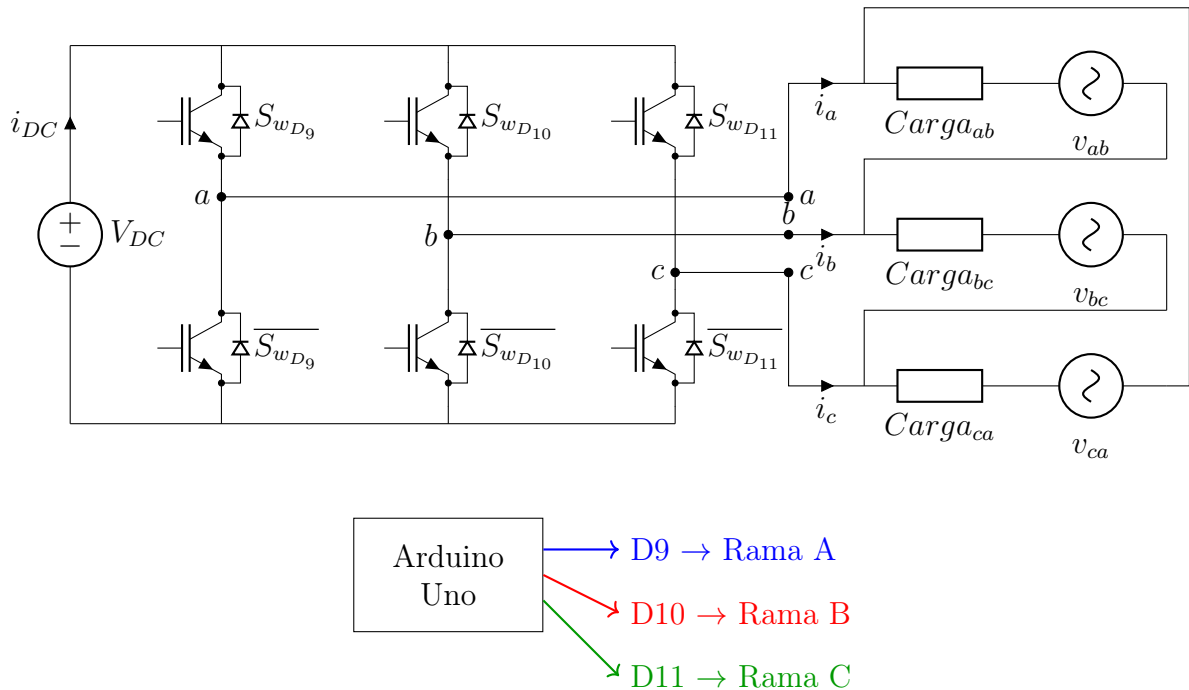


Figura 5.1: Esquema general del inversor trifasico controlado por Arduino Uno.

5.2. Estrategias de Modulacion para Trifasico

5.2.1. Modulacion Vectorial (SVPWM)

La SVPWM trata el conjunto de tensiones como un vector espacial. El inversor tiene 8 estados de conmutacion (6 activos + 2 nulos). El vector de referencia se sintetiza promediando los dos vectores activos adyacentes y uno de los nulos durante el periodo de conmutacion. La Figura ?? muestra el hexagono de vectores espaciales.

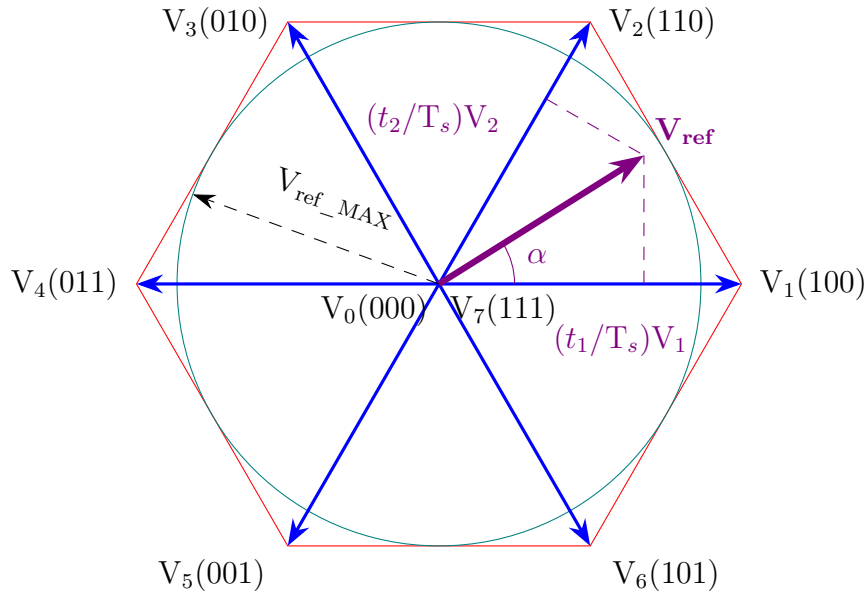


Figura 5.2: Hexágono de vectores espaciales y síntesis de $\vec{V}_{ref}$ en el Sector I.

5.2.2. Modulación Discontinua (DPWM)

La DPWM busca optimizar la eficiencia térmica enclavando alternadamente cada fase al riel positivo o negativo durante intervalos de 60° acumulados. La Figura ?? muestra la onda moduladora característica.

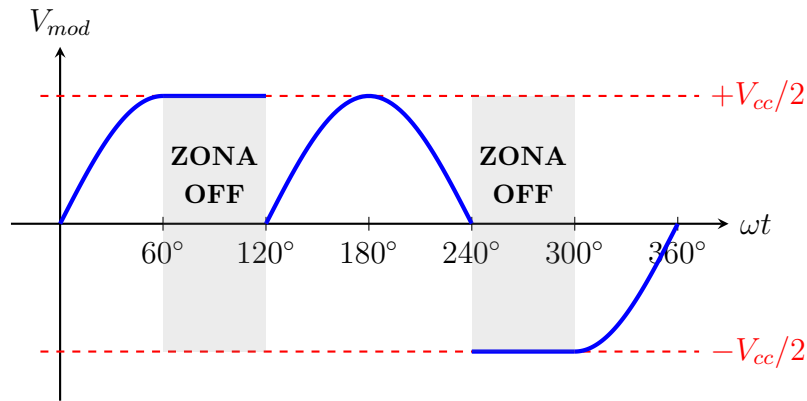


Figura 5.3: Onda moduladora resultante con la estrategia DPWM1 y sus áreas de descanso.

5.2.3. Modulación Generalizada de Vectores Espaciales

La modulación generalizada unifica todas las estrategias mediante el parámetro δ , que controla la distribución de los vectores nulos. La Tabla ?? muestra cómo obtener distintas estrategias.

Tabla 5.1: Valores de δ para distintas estrategias de modulacion.

Estrategia	δ
DPWM _{mín}	1
DPWM _{máx}	0
SVPWM clasico	1/2
DPWM ₀	$\frac{1}{2}[1 + (-1)^{n_1}]$
DPWM ₁	$\frac{1}{2}[1 + (-1)^{n_2}]$
DPWM ₂	$\frac{1}{2}[1 + (-1)^{n_1+1}]$
DPWM ₃	$\frac{1}{2}[1 + (-1)^{n_2+1}]$

La implementacion en coordenadas naturales simplifica el algoritmo:

$$D_k = (1 - \delta)(1 - v_{\text{máx}}) - \delta v_{\text{mín}} + v_{kN}, \quad k = a, b, c$$

5.2.4. Comparativa de Tecnicas Avanzadas

La Tabla ?? resume las características de las técnicas avanzadas.

Tabla 5.2: Comparativa de técnicas avanzadas para inversor trifasico.

Característica	SVPWM	DPWM	Histeresis	PWM Regular
Aprovechamiento V_{DC}	Alto (1.15)	Alto (1.15)	Variable	Bajo (1.0)
THD típica	Baja	Media	Media-Alta	Media
Perdidas conmutacion	Medias	Bajas	Altas	Medias
Complejidad calculo	Media	Media	Baja	Baja
Frecuencia conmutacion	Fija	Fija	Variable	Fija
Respuesta dinamica	Buena	Buena	Muy rapida	Buena

Capítulo 6

Comparativa General y Criterios de Seleccin

6.1. Resumen de Tecnicas de Modulacion

La Tabla ?? resume las características de las técnicas de modulación para inversores monofásicos y trifásicos.

Tabla 6.1: Resumen de técnicas de modulación.

Característica	SPWM	THIPWM	SVPWM	DPWM
Aprovechamiento V_{DC}	1.0	1.15	1.15	1.15
THD típica	Baja	Baja-Media	Baja	Media
Perdidas por conmutación	Medias	Medias	Medias	Bajas
Complejidad de cálculo	Baja	Media	Media	Media
Aplicación típica	General	Motores AC	Control vectorial	Alta potencia

6.2. Criterios de Seleccin

La selección de la técnica de modulación y del método de control digital depende de los siguientes factores:

- **Calidad de la forma de onda:** SPWM o SVPWM ofrecen baja THD.
- **Eficiencia energética:** DPWM u onda cuadrada reducen pérdidas por conmutación.
- **Respuesta dinámica:** Control por histéresis proporciona la respuesta más rápida.
- **Recursos del microcontrolador:** SPWM es más ligera que SVPWM.
- **Potencia de la carga:** Para altas potencias, se prioriza la reducción de pérdidas (onda cuadrada o DPWM).
- **Seguridad:** El método de conmutación atómico (Método 3) es el más seguro para evitar conducción cruzada.

6.3. Recomendaciones para la Implementación

1. Utilizar el Metodo 3 (asignación atómica por máscara) para la actualización de las salidas digitales.
2. Generar los tiempos muertos mediante hardware (drivers de compuerta con protección integrada) o mediante software con temporizadores precisos.
3. Para aplicaciones de baja potencia y fines educativos, SPWM con tablas precalculadas ofrece un buen equilibrio.
4. Para aplicaciones de alta eficiencia, implementar SVPWM o DPWM con el algoritmo generalizado basado en δ .
5. Almacenar las tablas de modulación en la memoria flash (PROGMEM) para ahorrar RAM.

Capítulo 7

Conclusiones Generales

El presente documento ha integrado cuatro trabajos técnicos que abordan de manera exhaustiva el diseño, análisis e implementación de inversores de potencia controlados por la plataforma Arduino Uno.

El análisis de la etapa de medio puente con driver bootstrap discreto ha demostrado que es posible generar tiempos muertos mediante redes RC con valores típicos de $R_B = 2,2\text{ k}\Omega$ y $C_{DEAD} = 1\text{ nF}$, logrando retardos de aproximadamente $4\text{ }\mu\text{s}$. El ciclo de trabajo máximo alcanzable es del 96 %, aunque en la práctica se recomienda limitarlo al 90 % para garantizar la recarga del bootstrap. Las pérdidas totales estimadas son del orden de 190 mW para una corriente de carga de 1 A .

En cuanto a los métodos de control digital, la comparación de las tres técnicas presentadas muestra que el Método 3 (asignación atómica por máscara) es el más adecuado para aplicaciones de electrónica de potencia, ya que ofrece la máxima velocidad (187 ns) y atomicidad, eliminando el riesgo de estados transitorios inseguros. Su principal desventaja es la dependencia de la arquitectura y la menor legibilidad, aspectos que se pueden mitigar mediante comentarios y encapsulamiento en funciones.

Las técnicas de modulación para inversores monofásicos (SPWM, THIPWM, MPWM y onda cuadrada) han sido descritas con sus respectivas ventajas y desventajas. La SPWM ofrece un buen equilibrio entre simplicidad y calidad de la onda, mientras que la THIPWM y la MPWM permiten mejorar el aprovechamiento del bus DC. La modulación por onda cuadrada, aunque presenta una THD elevada (48%), minimiza las pérdidas por conmutación, siendo adecuada para aplicaciones de muy alta potencia.

Para inversores trifásicos, las estrategias avanzadas como SVPWM y DPWM ofrecen un mayor aprovechamiento del bus DC (hasta 1.15) y, en el caso de DPWM, una reducción significativa de las pérdidas por conmutación. La modulación generalizada de vectores espaciales, basada en el parámetro δ , proporciona un marco unificado que permite implementar todas las estrategias con un único algoritmo, facilitando la experimentación y la optimización según los requisitos de la aplicación.

La implementación práctica en Arduino Uno, utilizando el Timer1 en modo Fast PWM con TOP programable por ICR1, ha sido detallada con ejemplos de código y configuraciones típicas. Se ha destacado la importancia de generar tiempos muertos adecuados y de dimensionar correctamente los componentes de la etapa de potencia.

En conjunto, el documento proporciona una base sólida para el diseño de inversores de potencia controlados por Arduino, abarcando desde la etapa de potencia hasta las estrategias de modulación más avanzadas. Se recomienda que, en aplicaciones reales, se incorporen circuitos de protección adicionales (bloqueo por subtensión, protección térmica,

limitación de corriente) y se verifique la compatibilidad de los tiempos de conmutación de los dispositivos de potencia con los tiempos muertos programados.

Apéndice A

Anexos: Codigos de Ejecucion

A.1. Anexo A: Codigo Arduino para Generacion de SPWM con Timer1

El siguiente codigo implementa la modulacion SPWM para un inversor monofasico puente H utilizando el Timer1 del ATmega328P. Las tablas de modulacion se almacenan en la memoria flash (PROGMEM).

```
1 #include <avr/pgmspace.h>
2 #include <avr/interrupt.h>
3
4 // Numero de muestras por periodo (m_f)
5 #define N_SAMPLES 60
6
7 // Valor de ICR1 para la frecuencia de portadora deseada (ejemplo: 720
  Hz con prescaler 8)
8 #define ICR1_VALUE 3332
9
10 // Tiempo muerto en ticks (ejemplo: 4 ticks para ~2 us)
11 #define DEAD_TICKS 4
12
13 // Tablas de modulacion para las ramas A y B (almacenadas en PROGMEM)
14 const uint16_t tablaA[N_SAMPLES] PROGMEM = {
15     // Valores calculados para SPWM con ma = 0.8, mf = 12, ICR1 = 3332
16     // ... (se generan con Python)
17 };
18
19 const uint16_t tablaB[N_SAMPLES] PROGMEM = {
20     // Valores complementarios
21     // ...
22 };
23
24 volatile uint8_t idx = 0;
25
26 // Rutina de servicio de interrupcion por overflow del Timer1
27 ISR(TIMER1_OVF_vect) {
28     idx++;
29     if (idx >= N_SAMPLES) idx = 0;
30     OCR1A = pgm_read_word(&tablaA[idx]);
31     OCR1B = pgm_read_word(&tablaB[idx]);
32 }
33
```

```

34 void setup() {
35     // Configurar pines 9 y 10 como salidas
36     DDRB |= (1 << PB1) | (1 << PB2);
37
38     // Configurar Timer1 en modo Fast PWM con TOP = ICR1
39     TCCR1A = 0;
40     TCCR1B = 0;
41     TCCR1A |= (1 << WGM11);           // Modo 14
42     TCCR1B |= (1 << WGM13) | (1 << WGM12);
43     TCCR1A |= (1 << COM1A1) | (1 << COM1B1); // Salida no invertida
44
45     // Seleccionar prescaler = 8
46     TCCR1B |= (1 << CS11);
47
48     // Establecer TOP
49     ICR1 = ICR1_VALUE;
50
51     // Cargar valores iniciales
52     OCR1A = pgm_read_word(&tablaA[0]);
53     OCR1B = pgm_read_word(&tablaB[0]);
54
55     // Habilitar interrupcion por overflow
56     TIMSK1 |= (1 << TOIE1);
57
58     // Habilitar interrupciones globales
59     sei();
60 }
61
62 void loop() {
63     // El control se realiza en la ISR
64     // Se puede agregar codigo para cambiar parametros dinamicamente
65 }

```

Listing A.1: Código Arduino para SPWM con Timer1

A.2. Anexo B: Código Python para Generación de Tablas de Modulación

El siguiente script en Python genera las tablas de modulación para SPWM, THIPWM y MPWM, exportandolas en formato adecuado para el código Arduino.

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import numpy as np
5  import math
6
7  # Parametros de modulación
8  f_ref = 60           # Frecuencia fundamental (Hz)
9  m_f = 12             # Índice de modulación de frecuencia
10 m_a = 0.8            # Índice de modulación de amplitud
11 f_c = m_f * f_ref    # Frecuencia de portadora (Hz)
12 ICR1 = 3332          # Valor de TOP (calculado para prescaler 8)
13 DEAD_TICKS = 4       # Tiempo muerto en ticks
14
15 # Numero de muestras

```

```

16 N = m_f
17
18 # Generacion de la referencia senoidal (SPWM)
19 t = np.linspace(0, 2*np.pi, N, endpoint=False)
20 ref_sin = m_a * np.sin(t)
21
22 # Generacion de la referencia con tercer armonico (THIPWM)
23 a3 = 0.25
24 ref_thi = m_a * (np.sin(t) + a3 * np.sin(3*t))
25 ref_thi = ref_thi / np.max(np.abs(ref_thi))
26
27 # Generacion de la referencia hiperbolica (MPWM)
28 k = 2
29 ref_tanh = m_a * np.tanh(k * np.sin(t))
30 ref_tanh = ref_tanh / np.max(np.abs(ref_tanh))
31
32 # Funcion para calcular los valores OCR
33 def calcular_OCR(ref, ICR1, dead_ticks):
34     duty_A = (1 + ref) / 2
35     duty_B = (1 - ref) / 2
36     ocr_A = np.round(duty_A * ICR1).astype(int)
37     ocr_B = np.round(duty_B * ICR1).astype(int)
38     # Aplicar tiempo muerto
39     ocr_A = np.clip(ocr_A, dead_ticks, ICR1 - dead_ticks)
40     ocr_B = np.clip(ocr_B, dead_ticks, ICR1 - dead_ticks)
41     return ocr_A, ocr_B
42
43 # Calcular para SPWM
44 ocr_A_sin, ocr_B_sin = calcular_OCR(ref_sin, ICR1, DEAD_TICKS)
45
46 # Imprimir tabla en formato Arduino
47 print("// Tabla para SPWM (OCR1A)")
48 print("const uint16_t tablaA[N_SAMPLES] PROGMEM = {")
49 for i, val in enumerate(ocr_A_sin):
50     if i % 8 == 0:
51         print("    ", end="")
52     print(f"{val:4d}, ", end="")
53     if (i+1) % 8 == 0 or i == N-1:
54         print()
55 print("};")
56
57 print("\n// Tabla para SPWM (OCR1B)")
58 print("const uint16_t tablaB[N_SAMPLES] PROGMEM = {")
59 for i, val in enumerate(ocr_B_sin):
60     if i % 8 == 0:
61         print("    ", end="")
62     print(f"{val:4d}, ", end="")
63     if (i+1) % 8 == 0 or i == N-1:
64         print()
65 print("};")

```

Listing A.2: Codigo Python para generar tablas de modulacion

A.3. Anexo C: Código Arduino para SVPWM Generalizado

El siguiente código muestra la implementación del algoritmo generalizado de SVPWM para inversor trifásico.

```

1 // Variables globales
2 volatile float Va_ref, Vb_ref, Vc_ref; // Tensiones de referencia
   normalizadas
3 volatile uint8_t sector;
4 volatile uint16_t Ta, Tb, Tc;
5
6 // Funcion para determinar el sector
7 uint8_t getSector(float vx, float vy) {
8     // vx, vy en coordenadas estacionarias (alfa, beta)
9     // Usar expresiones con signo para evitar atan2
10    float fy = vy / sqrt(3.0);
11    float fx = vx;
12    int N = (int)(2.5 - signbit(fy) * ((fx > fy) + (fx > -fy) + 0.5));
13    return (uint8_t)(N % 6);
14 }
15
16 // Funcion principal de SVPWM
17 void computeSVPWM(float vx, float vy, float delta) {
18     // vx, vy normalizadas respecto a Vdc/2
19    float fy = vy / sqrt(3.0);
20    float fx = vx;
21
22    // Obtener sector
23    sector = getSector(fx, fy);
24
25    // Calcular tiempos de los vectores activos (en por unidad de T_s)
26    float T1, T2;
27    switch(sector) {
28        case 0: // Sector I
29            T1 = fx - fy;
30            T2 = 2.0 * fy;
31            break;
32        case 1: // Sector II
33            T1 = fx + fy;
34            T2 = -fx + fy;
35            break;
36        case 2: // Sector III
37            T1 = 2.0 * fy;
38            T2 = -fx - fy;
39            break;
40        case 3: // Sector IV
41            T1 = -fx + fy;
42            T2 = -2.0 * fy;
43            break;
44        case 4: // Sector V
45            T1 = -fx - fy;
46            T2 = fx - fy;
47            break;
48        case 5: // Sector VI
49            T1 = -2.0 * fy;
50            T2 = fx + fy;

```



```

51         break;
52     default:
53         T1 = T2 = 0;
54 }
55
56 // Limitar T1, T2 entre 0 y 1
57 T1 = constrain(T1, 0.0, 1.0);
58 T2 = constrain(T2, 0.0, 1.0);
59
60 // Calcular tiempos de los vectores nulos (en por unidad)
61 float T0 = 1.0 - T1 - T2;
62 if (T0 < 0) T0 = 0;
63
64 // Distribucion de los vectores nulos segun delta
65 // T0_000 = delta * T0, T0_111 = (1-delta) * T0
66
67 // Asignar tiempos a las ramas segun el sector
68 // (Implementacion simplificada; ver referencias para casos
completos)
69 float duty_a, duty_b, duty_c;
70 switch(sector) {
71     case 0:
72         duty_a = T1 + T2;
73         duty_b = T2;
74         duty_c = 0;
75         break;
76     case 1:
77         duty_a = T1;
78         duty_b = T1 + T2;
79         duty_c = 0;
80         break;
81     case 2:
82         duty_a = 0;
83         duty_b = T1 + T2;
84         duty_c = T2;
85         break;
86     case 3:
87         duty_a = 0;
88         duty_b = T1;
89         duty_c = T1 + T2;
90         break;
91     case 4:
92         duty_a = T2;
93         duty_b = 0;
94         duty_c = T1 + T2;
95         break;
96     case 5:
97         duty_a = T1 + T2;
98         duty_b = 0;
99         duty_c = T1;
100        break;
101    default:
102        duty_a = duty_b = duty_c = 0;
103 }
104
105 // Escalar a valores de OCR (considerando ICR1)
106 Ta = (uint16_t)(duty_a * ICR1);
107 Tb = (uint16_t)(duty_b * ICR1);

```

```

108     Tc = (uint16_t)(duty_c * ICR1);
109
110     // Aplicar tiempo muerto (dead_ticks)
111     Ta = constrain(Ta, DEAD_TICKS, ICR1 - DEAD_TICKS);
112     Tb = constrain(Tb, DEAD_TICKS, ICR1 - DEAD_TICKS);
113     Tc = constrain(Tc, DEAD_TICKS, ICR1 - DEAD_TICKS);
114
115     // Actualizar registros OCR (suponiendo Timer1 para A y B, y Timer2
116     para C)
117     OCR1A = Ta;
118     OCR1B = Tb;
119     // Para D11 (PB3) se puede usar Timer2 o actualizar por software en
120     la ISR
121     // OCR2A = Tc; (si se configura Timer2)
122 }
123 // ISR para actualizar los valores de comparacion
124 ISR(TIMER1_OVF_vect) {
125     // Calcular referencias en el dominio alfa-beta a partir de las
126     fases
127     // (Se asume que Va_ref, Vb_ref, Vc_ref ya estan actualizadas)
128     float vx = Va_ref - 0.5 * (Vb_ref + Vc_ref);
129     float vy = 0.866 * (Vb_ref - Vc_ref);
130
131     // Elegir delta segun estrategia deseada (ej. SVPWM clasico: delta =
132     0.5)
133     float delta = 0.5;
134
135     computeSVPWM(vx, vy, delta);
136 }

```

Listing A.3: Código Arduino para SVPWM generalizado

Apéndice B

Anexos: Códigos de Práctica Monofásico

B.1. Inversor Monofásico

Este programa controla el inversor Monofásico

```
1  /*
2   Generador de onda de 50/60 Hz con selección de método de disparo.
3   Pines: 9 (PB1) y 10 (PB2) como salidas complementarias.
4   Comandos por serial:
5       F50  -> frecuencia 50 Hz
6       F60  -> frecuencia 60 Hz
7       M1   -> método 1 (digitalWrite)
8       M2   -> método 2 (registros DDRB/PORTB con bits)
9       M3   -> método 3 (asignación directa con máscara)
10      H     -> muestra ayuda
11  */
12
13  // Variables de configuración
14  int frecuencia = 50;           // 50 o 60 Hz
15  int metodo = 1;               // 1, 2 o 3
16  unsigned long semiPeriodoUs; // duración de medio período en
    microsegundos
17
18  // Estado de la salida
19  bool d9High = true;           // true = D9 HIGH, D10 LOW; false = D9 LOW
    , D10 HIGH
20  unsigned long tiempoAnterior = 0;
21
22  // ----- Funciones de escritura según método -----
23  void setPinsMethod1(bool d9High) {
24      if (d9High) {
25          digitalWrite(9, HIGH);
26          digitalWrite(10, LOW);
27      } else {
28          digitalWrite(9, LOW);
29          digitalWrite(10, HIGH);
30      }
31  }
32
33  void setPinsMethod2(bool d9High) {
34      if (d9High) {
```

```

35     PORTB |= (1 << PB1);          // D9 HIGH
36     PORTB &= ~(1 << PB2);        // D10 LOW
37 } else {
38     PORTB &= ~(1 << PB1);        // D9 LOW
39     PORTB |= (1 << PB2);         // D10 HIGH
40 }
41 }
42
43 void setPinsMethod3(bool d9High) {
44     if (d9High) {
45         // Poner D9 HIGH, D10 LOW sin afectar otros bits de PORTB
46         PORTB = (PORTB & ~((1 << PB1) | (1 << PB2))) | (1 << PB1);
47     } else {
48         PORTB = (PORTB & ~((1 << PB1) | (1 << PB2))) | (1 << PB2);
49     }
50 }
51
52 // Función que llama al método seleccionado
53 void aplicarMetodo(bool d9High) {
54     switch (metodo) {
55         case 1: setPinsMethod1(d9High); break;
56         case 2: setPinsMethod2(d9High); break;
57         case 3: setPinsMethod3(d9High); break;
58         default: setPinsMethod1(d9High); break;
59     }
60 }
61
62 // ----- Actualización de la frecuencia -----
63 void actualizarFrecuencia(int freq) {
64     frecuencia = freq;
65     if (frecuencia == 50) {
66         semiPeriodoUs = 10000;      // 10 ms
67     } else if (frecuencia == 60) {
68         semiPeriodoUs = 8333;       // 8.333 ms (aproximado)
69     } else {
70         semiPeriodoUs = 10000;      // fallback
71     }
72     // Reiniciamos la temporización para evitar transiciones bruscas
73     tiempoAnterior = micros();
74 }
75
76 // ----- Configuración inicial -----
77 void setup() {
78     Serial.begin(9600);
79     while (!Serial) { ; } // esperar conexión (opcional)
80
81     // Configurar pines 9 y 10 como salida
82     pinMode(9, OUTPUT);
83     pinMode(10, OUTPUT);
84     // También configurar DDRB por si se usa método 2 o 3
85     DDRB |= (1 << PB1) | (1 << PB2);
86
87     // Inicializar con 50 Hz y método 1
88     actualizarFrecuencia(50);
89     metodo = 1;
90     d9High = true;
91     aplicarMetodo(d9High);
92     tiempoAnterior = micros();

```

```
93
94 // Mostrar mensaje de bienvenida
95 Serial.println("Generador de onda 50/60 Hz");
96 Serial.println("Comandos: F50, F60, M1, M2, M3, H");
97 Serial.print("Frecuencia actual: ");
98 Serial.print(frecuencia);
99 Serial.println(" Hz");
100 Serial.print("Método actual: ");
101 Serial.println(metodo);
102 }
103
104 // ----- Bucle principal -----
105 void loop() {
106 // 1. Manejo de comandos seriales
107 if (Serial.available() > 0) {
108     String comando = Serial.readStringUntil('\n');
109     comando.trim();
110     comando.toUpperCase();
111
112     if (comando == "F50") {
113         actualizarFrecuencia(50);
114         Serial.println("Frecuencia cambiada a 50 Hz");
115     } else if (comando == "F60") {
116         actualizarFrecuencia(60);
117         Serial.println("Frecuencia cambiada a 60 Hz");
118     } else if (comando == "M1") {
119         metodo = 1;
120         Serial.println("Método cambiado a 1 (digitalWrite)");
121     } else if (comando == "M2") {
122         metodo = 2;
123         Serial.println("Método cambiado a 2 (registros)");
124     } else if (comando == "M3") {
125         metodo = 3;
126         Serial.println("Método cambiado a 3 (máscara)");
127     } else if (comando == "H") {
128         Serial.println("Comandos disponibles:");
129         Serial.println(" F50 -> Frecuencia 50 Hz");
130         Serial.println(" F60 -> Frecuencia 60 Hz");
131         Serial.println(" M1 -> Método 1 (digitalWrite)");
132         Serial.println(" M2 -> Método 2 (registros)");
133         Serial.println(" M3 -> Método 3 (máscara)");
134         Serial.println(" H -> Mostrar esta ayuda");
135         Serial.print("Estado actual: ");
136         Serial.print(frecuencia);
137         Serial.print(" Hz, Método ");
138         Serial.println(metodo);
139     } else {
140         Serial.println("Comando no reconocido. Envíe H para ayuda.");
141     }
142 }
143
144 // 2. Generación de la onda (no bloqueante)
145 unsigned long ahora = micros();
146 if (ahora - tiempoAnterior >= semiPeriodoUs) {
147     tiempoAnterior = ahora;
148     // Cambiar estado
149     d9High = !d9High;
150     aplicarMetodo(d9High);
```

```

151 }
152 }

```

Este programa es la interfaz grafica del programa anterior

```

1 import tkinter as tk
2 from tkinter import ttk, scrolledtext, messagebox
3 import serial
4 import serial.tools.list_ports
5 import threading
6 import queue
7 import time
8
9 class SerialController:
10     """Maneja la conexión serie y la comunicación en un hilo separado.
11     """
12     def __init__(self, update_text_callback):
13         self.serial_port = None
14         self.connected = False
15         self.read_thread = None
16         self.stop_event = threading.Event()
17         self.update_text = update_text_callback # función para agregar
18         texto a la GUI
19         self.write_queue = queue.Queue() # cola para comandos a enviar
20
21     def connect(self, port, baudrate=9600):
22         """Intenta conectar al puerto especificado."""
23         if self.connected:
24             self.disconnect()
25         try:
26             self.serial_port = serial.Serial(port, baudrate, timeout
27             =0.1)
28             self.connected = True
29             self.stop_event.clear()
30             # Iniciar hilo de lectura
31             self.read_thread = threading.Thread(target=self._read_loop,
32             daemon=True)
33             self.read_thread.start()
34             # Iniciar hilo de escritura
35             self.write_thread = threading.Thread(target=self._write_loop
36             , daemon=True)
37             self.write_thread.start()
38             return True
39         except Exception as e:
40             self.update_text(f"Error al conectar: {e}")
41             return False
42
43     def disconnect(self):
44         """Cierra la conexión serie."""
45         if self.connected:
46             self.stop_event.set()
47             if self.read_thread and self.read_thread.is_alive():
48                 self.read_thread.join(timeout=1)
49             if self.write_thread and self.write_thread.is_alive():
50                 self.write_thread.join(timeout=1)
51             if self.serial_port and self.serial_port.is_open:
52                 self.serial_port.close()
53             self.connected = False
54             self.serial_port = None

```

```

50         self.update_text("Desconectado")
51
52     def send_command(self, command):
53         """Envía un comando a través de la cola de escritura."""
54         if self.connected:
55             self.write_queue.put(command + '\n')
56         else:
57             self.update_text("No conectado. No se puede enviar el
comando.")
58
59     def _read_loop(self):
60         """Bucle de lectura en hilo separado."""
61         while not self.stop_event.is_set():
62             if self.serial_port and self.serial_port.is_open:
63                 try:
64                     if self.serial_port.in_waiting > 0:
65                         line = self.serial_port.readline().decode('utf-8
', errors='ignore').strip()
66                         if line:
67                             self.update_text(f"Arduino: {line}")
68                 except Exception as e:
69                     self.update_text(f"Error de lectura: {e}")
70                     break
71             time.sleep(0.01)
72
73     def _write_loop(self):
74         """Bucle de escritura desde la cola."""
75         while not self.stop_event.is_set():
76             try:
77                 command = self.write_queue.get(timeout=0.1)
78                 if self.serial_port and self.serial_port.is_open:
79                     self.serial_port.write(command.encode())
80             except queue.Empty:
81                 continue
82             except Exception as e:
83                 self.update_text(f"Error al escribir: {e}")
84                 break
85
86
87 class App:
88     def __init__(self, root):
89         self.root = root
90         self.root.title("Controlador de Generador de Onda - Arduino")
91         self.root.geometry("600x500")
92
93         # Variables de control
94         self.freq_var = tk.StringVar(value="50")
95         self.method_var = tk.StringVar(value="1")
96         self.port_var = tk.StringVar()
97
98         # Inicializar controlador serie con callback de actualización
99         self.serial_ctrl = SerialController(self.update_text)
100
101         # Crear interfaz
102         self.create_widgets()
103
104         # Actualizar lista de puertos al iniciar
105         self.refresh_ports()

```

```

106
107     # Manejar cierre de ventana
108     self.root.protocol("WM_DELETE_WINDOW", self.on_close)
109
110     def create_widgets(self):
111         # Frame superior: puerto y conexión
112         top_frame = ttk.LabelFrame(self.root, text="Conexión Serie",
padding=5)
113         top_frame.pack(fill=tk.X, padx=10, pady=5)
114
115         ttk.Label(top_frame, text="Puerto:").grid(row=0, column=0, padx
=5, pady=5, sticky=tk.W)
116         self.port_combo = ttk.Combobox(top_frame, textvariable=self.
port_var, state="readonly", width=20)
117         self.port_combo.grid(row=0, column=1, padx=5, pady=5)
118         self.port_combo.bind('<<ComboboxSelected>>', self.
on_port_selected)
119
120         self.btn_refresh = ttk.Button(top_frame, text="Actualizar
puertos", command=self.refresh_ports)
121         self.btn_refresh.grid(row=0, column=2, padx=5, pady=5)
122
123         self.btn_connect = ttk.Button(top_frame, text="Conectar",
command=self.toggle_connection)
124         self.btn_connect.grid(row=0, column=3, padx=5, pady=5)
125
126         # Separador
127         ttk.Separator(self.root, orient='horizontal').pack(fill=tk.X,
padx=10, pady=5)
128
129         # Frame medio: configuración (frecuencia y método)
130         config_frame = ttk.LabelFrame(self.root, text="Configuración",
padding=10)
131         config_frame.pack(fill=tk.X, padx=10, pady=5)
132
133         # Frecuencia
134         freq_frame = ttk.Frame(config_frame)
135         freq_frame.pack(side=tk.LEFT, padx=10, fill=tk.X, expand=True)
136         ttk.Label(freq_frame, text="Frecuencia:").pack(anchor=tk.W)
137         ttk.Radiobutton(freq_frame, text="50 Hz", variable=self.freq_var
, value="50").pack(anchor=tk.W)
138         ttk.Radiobutton(freq_frame, text="60 Hz", variable=self.freq_var
, value="60").pack(anchor=tk.W)
139
140         # Método
141         method_frame = ttk.Frame(config_frame)
142         method_frame.pack(side=tk.LEFT, padx=10, fill=tk.X, expand=True)
143         ttk.Label(method_frame, text="Método de disparo:").pack(anchor=
tk.W)
144         ttk.Radiobutton(method_frame, text="Método 1 (digitalWrite)",
variable=self.method_var, value="1").pack(anchor=tk.W)
145         ttk.Radiobutton(method_frame, text="Método 2 (registros)",
variable=self.method_var, value="2").pack(anchor=tk.W)
146         ttk.Radiobutton(method_frame, text="Método 3 (máscara)",
variable=self.method_var, value="3").pack(anchor=tk.W)
147
148         # Botón aplicar
149         btn_apply = ttk.Button(config_frame, text="Aplicar configuración

```



```

", command=self.apply_config)
    btn_apply.pack(side=tk.RIGHT, padx=10, pady=10)

    # Frame inferior: área de texto (logs)
    log_frame = ttk.LabelFrame(self.root, text="Mensajes del Arduino",
padding=5)
    log_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)

    self.text_area = scrolledtext.ScrolledText(log_frame, height=15,
state='normal')
    self.text_area.pack(fill=tk.BOTH, expand=True)
    self.text_area.config(state='disabled')

    # Botón limpiar logs
    btn_clear = ttk.Button(self.root, text="Limpiar mensajes",
command=self.clear_logs)
    btn_clear.pack(pady=5)

    def refresh_ports(self):
        """Actualiza la lista de puertos disponibles."""
        ports = [port.device for port in serial.tools.list_ports.
comports()]
        self.port_combo['values'] = ports
        if ports:
            self.port_combo.set(ports[0])
        else:
            self.port_combo.set('')
            self.port_var.set('')

    def on_port_selected(self, event):
        """Se activa al seleccionar un puerto."""
        pass

    def toggle_connection(self):
        """Conecta o desconecta según el estado actual."""
        if self.serial_ctrl.connected:
            self.serial_ctrl.disconnect()
            self.btn_connect.config(text="Conectar")
            self.port_combo.config(state="readonly")
            self.btn_refresh.config(state="normal")
        else:
            port = self.port_var.get()
            if not port:
                messagebox.showwarning("Puerto no seleccionado", "Por
favor, seleccione un puerto.")
                return
            if self.serial_ctrl.connect(port):
                self.btn_connect.config(text="Desconectar")
                self.port_combo.config(state="disabled")
                self.btn_refresh.config(state="disabled")
                # Enviar estado actual al conectar (opcional)
                # No hay comando de consulta, pero podemos aplicar
configuración actual
                self.apply_config()
            else:
                self.btn_connect.config(text="Conectar")

    def apply_config(self):

```

```

201     """Envía los comandos correspondientes a la frecuencia y método
seleccionados."""
202     if not self.serial_ctrl.connected:
203         messagebox.showwarning("No conectado", "Conéctese al Arduino
primero.")
204         return
205
206     freq = self.freq_var.get()
207     method = self.method_var.get()
208
209     # Enviar comando de frecuencia
210     if freq == "50":
211         self.serial_ctrl.send_command("F50")
212     elif freq == "60":
213         self.serial_ctrl.send_command("F60")
214
215     # Enviar comando de método
216     if method == "1":
217         self.serial_ctrl.send_command("M1")
218     elif method == "2":
219         self.serial_ctrl.send_command("M2")
220     elif method == "3":
221         self.serial_ctrl.send_command("M3")
222
223     self.update_text("Configuración enviada.")
224
225     def update_text(self, message):
226         """Agrega un mensaje al área de texto (llamado desde cualquier
hilo)."""
227         # Se debe ejecutar en el hilo principal
228         self.root.after(0, self._append_text, message)
229
230     def _append_text(self, message):
231         """Inserta texto en el área de logs."""
232         self.text_area.config(state='normal')
233         self.text_area.insert(tk.END, message + "\n")
234         self.text_area.see(tk.END)
235         self.text_area.config(state='disabled')
236
237     def clear_logs(self):
238         """Limpia el área de texto."""
239         self.text_area.config(state='normal')
240         self.text_area.delete(1.0, tk.END)
241         self.text_area.config(state='disabled')
242
243     def on_close(self):
244         """Cierra la conexión y la ventana."""
245         self.serial_ctrl.disconnect()
246         self.root.destroy()
247
248
249 if __name__ == "__main__":
250     root = tk.Tk()
251     app = App(root)
252     root.mainloop()

```

B.2. Inversor Monofásico Modulación

Modulación Bipolar Modulación Unipolar

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  """
5  Generador PWM para inversor monofásico con puente H (Arduino Uno)
6  -----
7  Modulación UNIPOLAR con portadora triangular en [0,1].
8  - Referencias: ref_A = 0.5*(1 + ma*seno), ref_B = 0.5*(1 - ma*seno)
9  - D9 y D10 se activan cuando su referencia supera la portadora.
10 - Salida: Vout = D9 - D10 (+Vdc, 0, -Vdc)
11
12 Dependencias: numpy, matplotlib, tkinter
13 Autor: Prof. Alexander Bueno (modificado)
14 Fecha: 08/2026
15 """
16
17 import tkinter as tk
18 from tkinter import ttk, messagebox
19 import numpy as np
20 import matplotlib.pyplot as plt
21 import os
22
23 #
24 # =====
25 # MODULO DE GENERACION DE ONDAS DE REFERENCIA
26 # =====
27
28 def evaluar_referencia(tipo, theta, params):
29     """
30     Evalúa la forma de onda de referencia normalizada (sin escalar por
31     ma)
32     en los ángulos theta, devolviendo valores en [-1, 1].
33
34     Parámetros:
35         tipo (str): 'SPWM', 'THIPWM', 'MPWM' o 'Onda Cuadrada'
36         theta (array): ángulos en radianes
37         params (dict): parámetros adicionales según el tipo:
38             - 'THIPWM': {'a3': float} amplitud de la 3ra armónica (0..1)
39             - 'MPWM': {'k': float} factor de forma (1..10)
40
41     Retorna:
42         array: valores de referencia entre -1 y 1
43     """
44     if tipo == "SPWM":
45         ref = np.sin(theta)
46     elif tipo == "THIPWM":
47         a3 = params.get('a3', 0.25)
48         raw = np.sin(theta) + a3 * np.sin(3 * theta)
49         max_raw = np.max(np.abs(raw))
50         ref = raw / max_raw if max_raw > 0 else np.zeros_like(theta)
51     elif tipo == "MPWM":
52         k = params.get('k', 3.0)

```

```

50     raw = np.tanh(k * np.sin(theta))
51     max_raw = np.max(np.abs(raw))
52     ref = raw / max_raw if max_raw > 0 else np.zeros_like(theta)
53 elif tipo == "Onda Cuadrada":
54     ref = np.sign(np.sin(theta))
55 else:
56     raise ValueError(f"Tipo de onda desconocido: {tipo}")
57 return ref
58
59
60 def generar_referencia(tipo, f_ref, mf, params):
61     """
62     Genera la tabla de referencia normalizada (sin multiplicar por ma)
63     para un periodo de la portadora.
64     """
65     theta = np.linspace(0, 2 * np.pi, mf, endpoint=False)
66     ref = evaluar_referencia(tipo, theta, params)
67     return ref # en [-1, 1]
68
69
70 #
=====
71 # MODULO DE CALCULO DEL TEMPORIZADOR Y GENERACION DEL CODIGO ARDUINO
72 #
=====
73
74 def calcular_parametros_timer(f_ref, mf, dead_time_us, ma, ref_norm):
75     """
76     Calcula la configuración del Timer1 (prescaler, ICR1) y las tablas
77     OCR1A y OCR1B para modulación unipolar.
78     ref_norm está en [-1, 1].
79     """
80     F_CLK = 16000000
81     PRESCALER_OPTS = [1, 8, 64, 256, 1024]
82     f_c = mf * f_ref
83
84     # ---- Selección del mejor prescaler para Phase Correct PWM ----
85     best_prescaler = 1
86     best_ICR = 0
87     best_error = float('inf')
88     for p in PRESCALER_OPTS:
89         ICR = (F_CLK / (2 * p * f_c)) - 1
90         if 1 <= ICR <= 65535:
91             ICR_round = int(round(ICR))
92             f_c_actual = F_CLK / (2 * p * (ICR_round + 1))
93             error = abs(f_c_actual - f_c)
94             if error < best_error:
95                 best_error = error
96                 best_prescaler = p
97                 best_ICR = ICR_round
98     if best_error == float('inf'):
99         raise ValueError("No se encontró configuración válida para el
100 Timer1.")
101     f_c_actual = F_CLK / (2 * best_prescaler * (best_ICR + 1))
102
103     # ---- Tiempo muerto en ticks ----

```

```

103     dead_ticks = int(round(dead_time_us * 1e-6 * F_CLK / best_prescaler)
104 )
105     if dead_ticks > best_ICR // 2:
106         dead_ticks = best_ICR // 4
107         print(f"ADVERTENCIA: Tiempo muerto ajustado a {dead_ticks *
108 best_prescaler / F_CLK * 1e6:.2f} us")
109
110     # ---- Cálculo de OCR para modulación unipolar ----
111     # ref_norm en [-1,1], ma en [0,1]
112     d_A = 0.5 * (1 + ma * ref_norm)
113     d_B = 0.5 * (1 - ma * ref_norm)
114
115     OCR1A_raw = d_A * best_ICR
116     OCR1B_raw = d_B * best_ICR
117
118     # Aplicar tiempo muerto restando a ambas cuando están activas
119     OCR1A = np.clip(OCR1A_raw - dead_ticks, 0, best_ICR).astype(int)
120     OCR1B = np.clip(OCR1B_raw - dead_ticks, 0, best_ICR).astype(int)
121
122     return best_prescaler, best_ICR, f_c_actual, dead_ticks, OCR1A,
123     OCR1B, mf
124
125 def generar_archivo_ino(f_ref, ma, mf, dead_time_us, tipo, params,
126 prescaler, ICR, f_c_actual, dead_ticks,
127 OCR1A, OCR1B, N, rms=0.0, thd=0.0):
128
129     """
130     Escribe el archivo SPWM.ino en la raíz.
131     Incluye en el encabezado el RMS y THD calculados.
132     Configura Timer1 en modo Phase Correct PWM con TOP=ICR1,
133     ambos canales como no inversores (modulación unipolar).
134     """
135     ruta = "SPWM.ino"
136
137     with open(ruta, "w") as f:
138         f.write("//
139 =====\n")
140         f.write("// PWM para inversor monofásico con puente H (Arduino
141 Uno)\n")
142         f.write("// Generado automáticamente por Python\n")
143         f.write("// Modo: Phase Correct PWM con portadora triangular
144 [0,1]\n")
145         f.write("// MODULACIÓN UNIPOLAR\n")
146         f.write("//
147 =====\n")
148         f.write(f"// f_ref = {f_ref:.2f} Hz\n")
149         f.write(f"// ma = {ma:.3f}\n")
150         f.write(f"// mf = {mf}\n")
151         f.write(f"// Tipo = {tipo}\n")
152         if params:
153             f.write(f"// Params = {params}\n")
154         f.write(f"// f_c = {f_c_actual:.2f} Hz (real)\n")
155         f.write(f"// Prescaler = {prescaler}, ICR1 = {ICR}\n")
156         f.write(f"// Tmuerto = {dead_time_us:.1f} us ({dead_ticks} tics
157 )\n")
158         f.write(f"// Muestras = {N}\n")
159         f.write(f"// RMS = {rms:.4f} (tensión de salida D9-D10
160 normalizada)\n")

```

```

152     f.write(f"//   THD           = {thd:.2f}%\n")
153     f.write("//
=====
)

154
155     f.write("#include <avr/pgmspace.h>\n\n")
156
157     f.write("void setup() {\n")
158     f.write("    pinMode(9, OUTPUT);    // Rama A del puente H\n")
159     f.write("    pinMode(10, OUTPUT);    // Rama B del puente H\n\n")
160     f.write("    // ---- Configuración del Timer1 en modo Phase
Correct PWM con TOP = ICR1 ----\n")
161     f.write("    TCCR1A = 0;\n")
162     f.write("    TCCR1B = 0;\n")
163     f.write("    // Modo 10: Phase Correct PWM, TOP=ICR1\n")
164     f.write("    TCCR1A |= (1 << WGM11);\n")
165     f.write("    TCCR1B |= (1 << WGM13);\n")
166     f.write("    TCCR1B &= ~(1 << WGM12);\n")
167     f.write("    TCCR1A &= ~(1 << WGM10);\n\n")
168     f.write("    // OC1A: no inversor (COM1A1=1, COM1A0=0)\n")
169     f.write("    TCCR1A |= (1 << COM1A1);\n")
170     f.write("    TCCR1A &= ~(1 << COM1A0);\n")
171     f.write("    // OC1B: no inversor (COM1B1=1, COM1B0=0)  <--
MODULACIÓN UNIPOLAR\n")
172     f.write("    TCCR1A |= (1 << COM1B1);\n")
173     f.write("    TCCR1A &= ~(1 << COM1B0);\n\n")
174
175     f.write("    // ---- Prescaler ----\n")
176     if prescaler == 1:
177         f.write("    TCCR1B |= (1 << CS10);\n")
178     elif prescaler == 8:
179         f.write("    TCCR1B |= (1 << CS11);\n")
180     elif prescaler == 64:
181         f.write("    TCCR1B |= (1 << CS11) | (1 << CS10);\n")
182     elif prescaler == 256:
183         f.write("    TCCR1B |= (1 << CS12);\n")
184     elif prescaler == 1024:
185         f.write("    TCCR1B |= (1 << CS12) | (1 << CS10);\n")
186     f.write("\n")
187
188     f.write(f"    ICR1 = {ICR};\n")
189     f.write(f"    OCR1A = {OCR1A[0]};\n")
190     f.write(f"    OCR1B = {OCR1B[0]};\n\n")
191     f.write("    TIMSK1 |= (1 << TOIE1);\n")
192     f.write("}\n\n")
193
194     f.write(f"const int N = {N};\n\n")
195     f.write(f"const unsigned int OCR1A_table[N] PROGMEM = {\n")
196     for i, v in enumerate(OCR1A):
197         f.write(f"    {v}{'', ' if i < N-1 else ''}\n")
198     f.write("};\n\n")
199
200     f.write(f"const unsigned int OCR1B_table[N] PROGMEM = {\n")
201     for i, v in enumerate(OCR1B):
202         f.write(f"    {v}{'', ' if i < N-1 else ''}\n")
203     f.write("};\n\n")
204
205     f.write("volatile unsigned char index = 0;\n\n")

```

```

206     f.write("ISR(TIMER1_OVF_vect) {\n")
207     f.write("     index++; \n")
208     f.write("     if (index >= N) index = 0; \n")
209     f.write("     OCR1A = pgm_read_word(&OCR1A_table[index]); \n")
210     f.write("     OCR1B = pgm_read_word(&OCR1B_table[index]); \n")
211     f.write("}\n\n")
212
213     f.write("void loop() {\n")
214     f.write("     // Todo el control se realiza por interrupción\n")
215     f.write("}\n")
216
217     print(f"Archivo generado: {ruta}")
218
219
220 #
221 # =====
222 # MÓDULO DE SIMULACIÓN, CÁLCULO DE RMS Y THD, Y GRÁFICAS
223 # =====
224
225 def simular_senales(f_ref, ma, mf, dead_time_us, ref_norm,
226 pts_por_periodo=200):
227     """
228     Simula las señales PWM (D9, D10) y la tensión de salida (D9-D10)
229     para dos ciclos completos de la referencia.
230     Portadora triangular en [0,1], modulación unipolar.
231     ref_norm está en [-1,1].
232     """
233     f_c = mf * f_ref
234     T_c = 1.0 / f_c
235     num_periodos = 2 * mf
236     tiempo_total = num_periodos * T_c
237     n_pts = num_periodos * pts_por_periodo
238     t = np.linspace(0, tiempo_total, n_pts, endpoint=False)
239
240     # Portadora triangular en [0,1] (pico en fase=0.5, valle en 0 y 1)
241     fase = (t % T_c) / T_c
242     portadora = 2 * np.minimum(fase, 1 - fase) # 0..1
243
244     # Referencia repetida
245     ref_norm_repetida = np.repeat(np.tile(ref_norm, 2), pts_por_periodo)
246
247     # Referencias para cada rama
248     ref_A = 0.5 * (1 + ma * ref_norm_repetida)
249     ref_B = 0.5 * (1 - ma * ref_norm_repetida)
250
251     # Comparación con la portadora
252     outA = (ref_A > portadora).astype(float)
253     outB = (ref_B > portadora).astype(float)
254
255     # Tensión de salida (normalizada)
256     diff = outA - outB # +1, 0, -1
257
258     return t, outA, outB, diff

```

```

259 def calcular_rms_thd(t, senal, f_ref):
260     """
261     Calcula el RMS total, el RMS de la fundamental y el THD (%).
262     """
263     dt = t[1] - t[0]
264     N = len(senal)
265     y = senal - np.mean(senal)
266     V_rms = np.sqrt(np.mean(y**2))
267     if V_rms < 1e-12:
268         return 0.0, 0.0, 0.0
269
270     Y = np.fft.rfft(y)
271     freqs = np.fft.rfftfreq(N, dt)
272     idx_fund = np.argmin(np.abs(freqs - f_ref))
273     if idx_fund == 0:
274         V1_peak = Y[0] / N
275     else:
276         V1_peak = 2 * np.abs(Y[idx_fund]) / N
277     V1_rms = V1_peak / np.sqrt(2)
278
279     max_armonica = 50
280     idx_max = min(len(Y)-1, int(max_armonica * f_ref / (freqs[1] - freqs
281     suma_cuadrados = 0.0
282     for k in range(1, idx_max+1):
283         if k == idx_fund:
284             continue
285         if k == 0:
286             Vk_peak = Y[0] / N
287         else:
288             Vk_peak = 2 * np.abs(Y[k]) / N
289         suma_cuadrados += (Vk_peak / np.sqrt(2))**2
290     thd = np.sqrt(suma_cuadrados) / V1_rms if V1_rms > 1e-12 else 0.0
291     return V_rms, V1_rms, thd * 100
292
293
294 def graficar_simulacion(f_ref, ma, mf, dead_time_us, ref_norm, tipo,
295     params):
296     """
297     Muestra tres subplots en vertical:
298     1) Referencias A y B + portadora triangular
299     2) PWM de D9 y D10 desplazadas verticalmente
300     3) Tensión de salida (D9 - D10)
301     Retorna (RMS, THD).
302     """
303     # Simulación
304     t, outA, outB, diff = simular_senales(f_ref, ma, mf, dead_time_us,
305     ref_norm, pts_por_periodo=200)
306     try:
307         V_rms, _, thd = calcular_rms_thd(t, diff, f_ref)
308     except Exception as e:
309         print(f"Error en cálculo de RMS/THD: {e}")
310         V_rms, thd = 0.0, 0.0
311
312     # Referencia y portadora de alta resolución para la gráfica
313     f_c = mf * f_ref
314     T_c = 1.0 / f_c
315     num_periodos = 2 * mf

```



```

314 tiempo_total = num_periodos * T_c
315 pts_high = 1000 * num_periodos
316 t_high = np.linspace(0, tiempo_total, pts_high, endpoint=False)
317 theta_high = 2 * np.pi * f_ref * t_high
318 ref_high = evaluar_referencia(tipo, theta_high, params) # en
[-1,1]
319 ref_A_high = 0.5 * (1 + ma * ref_high)
320 ref_B_high = 0.5 * (1 - ma * ref_high) # Referencia para D10
321 # Portadora triangular en [0,1]
322 fase_high = (t_high % T_c) / T_c
323 portadora_high = 2 * np.minimum(fase_high, 1 - fase_high)
324
325 # Crear figura con 3 subplots verticales
326 fig, axs = plt.subplots(3, 1, figsize=(14, 12))
327 fig.suptitle(
328     f'PWM con portadora triangular [0,1] - Modulación UNIPOLAR\n'
329     f'f_ref = {f_ref:.1f} Hz, ma = {ma:.2f}, mf = {mf}, Tmuerto = {
dead_time_us:.1f} us\n'
330     f'Tipo: {tipo} | RMS = {V_rms:.4f} | THD = {thd:.2f}%',
331     fontsize=14
332 )
333
334 # ---- Subplot 1: Referencias A y B + portadora ----
335 axs[0].plot(t_high, ref_A_high, 'b', lw=1.5, label='Referencia A (D9)')
336 axs[0].plot(t_high, ref_B_high, 'orange', lw=1.5, label='Referencia B (D10)')
337 axs[0].plot(t_high, portadora_high, 'r', lw=0.6, label='Portadora')
338 axs[0].axhline(y=0.5, color='gray', linestyle='--', linewidth=0.5)
339 axs[0].set_ylabel('Amplitud')
340 axs[0].grid(True)
341 axs[0].legend()
342 axs[0].set_title('Referencias A y B + Portadora (alta resolución)')
343 axs[0].set_xlim(t_high[0], t_high[-1])
344
345 # ---- Subplot 2: PWM D9 y D10 desplazadas ----
346 offset = 1.0
347 axs[1].plot(t, outA + offset, 'g', drawstyle='steps-post', lw=0.8,
label='PWM D9 (desplazado +1)')
348 axs[1].plot(t, outB - offset, 'm', drawstyle='steps-post', lw=0.8,
label='PWM D10 (desplazado -1)')
349 axs[1].axhline(y=offset, color='gray', linestyle=':', linewidth=0.5)
350 axs[1].axhline(y=-offset, color='gray', linestyle=':', linewidth
=0.5)
351 axs[1].axhline(y=0, color='k', linestyle='--', linewidth=0.5)
352 axs[1].set_ylabel('Nivel desplazado')
353 axs[1].set_xlabel('Tiempo (s)')
354 axs[1].grid(True)
355 axs[1].legend()
356 axs[1].set_title('Señales PWM en D9 y D10 (desplazadas verticalmente)')
357 axs[1].set_xlim(t[0], t[-1])
358
359 # ---- Subplot 3: Tensión de salida ----
360 axs[2].plot(t, diff, 'k', drawstyle='steps-post', lw=0.8)
361 axs[2].axhline(y=0, color='gray', linestyle='--', linewidth=0.5)
362 axs[2].set_ylabel('Tensión')
363 axs[2].set_xlabel('Tiempo (s)')

```

```

364     axs[2].grid(True)
365     axs[2].set_title('Tensión de salida (D9 - D10)')
366     axs[2].set_xlim(t[0], t[-1])
367     axs[2].set_ylim(-1.2, 1.2)
368
369     plt.tight_layout()
370     plt.show()
371     return V_rms, thd
372
373
374 #
=====
375 # INTERFAZ GRÁFICA (Tkinter)
376 #
=====
377
378 class SPWMApp:
379     """Aplicación principal con interfaz gráfica."""
380
381     def __init__(self, root):
382         self.root = root
383         root.title("Generador PWM para Arduino - Puente H (UNIPOLAR)")
384         root.geometry("600x580")
385         root.resizable(False, False)
386
387         # Variables de los parámetros
388         self.f_ref = tk.StringVar(value="60.0")
389         self.ma = tk.StringVar(value="0.8")
390         self.mf = tk.StringVar(value="12")
391         self.dead = tk.StringVar(value="2.0")
392         self.tipo = tk.StringVar(value="SPWM")
393         self.param_a3 = tk.StringVar(value="0.25")
394         self.param_k = tk.StringVar(value="3.0")
395
396         # Variables para mostrar resultados
397         self.rms_label = tk.StringVar(value="RMS: ---")
398         self.thd_label = tk.StringVar(value="THD: ---%")
399
400         self._crear_widgets()
401
402     def _crear_widgets(self):
403         """Construye la interfaz de usuario."""
404         root = self.root
405         row = 0
406
407         tk.Label(root, text="Frecuencia de referencia (Hz):").grid(row=
row, column=0, padx=10, pady=5, sticky='e')
408         tk.Entry(root, textvariable=self.f_ref, width=15).grid(row=row,
column=1, padx=10, pady=5, sticky='w')
409         row += 1
410
411         tk.Label(root, text="Índice de modulación ma (0-1):").grid(row=
row, column=0, padx=10, pady=5, sticky='e')
412         tk.Entry(root, textvariable=self.ma, width=15).grid(row=row,
column=1, padx=10, pady=5, sticky='w')
413         row += 1

```

```

414         tk.Label(root, text="Índice de modulación mf (entero):").grid(
415             row=row, column=0, padx=10, pady=5, sticky='e')
416         tk.Entry(root, textvariable=self.mf, width=15).grid(row=row,
417             column=1, padx=10, pady=5, sticky='w')
418         row += 1
419
420         tk.Label(root, text="Tiempo muerto (us):").grid(row=row, column
421             =0, padx=10, pady=5, sticky='e')
422         tk.Entry(root, textvariable=self.dead, width=15).grid(row=row,
423             column=1, padx=10, pady=5, sticky='w')
424         row += 1
425
426         tk.Label(root, text="Tipo de referencia:").grid(row=row, column
427             =0, padx=10, pady=5, sticky='e')
428         self.tipo_combo = ttk.Combobox(root, textvariable=self.tipo,
429             values=["SPWM", "THIPWM", "MPWM",
430                 "Onda Cuadrada"],
431                 state="readonly")
432         self.tipo_combo.grid(row=row, column=1, padx=10, pady=5, sticky=
433             'w')
434         self.tipo_combo.bind("<<ComboboxSelected>>", self.
435             _actualizar_parametros)
436         row += 1
437
438         self.frame_params = tk.Frame(root)
439         self.frame_params.grid(row=row, column=0, columnspan=2, pady=5)
440         self._actualizar_parametros()
441         row += 1
442
443         tk.Label(root, textvariable=self.rms_label, font=("Arial", 11, "
444             bold"), fg="blue").grid(row=row, column=0, padx=10, pady=5)
445         tk.Label(root, textvariable=self.thd_label, font=("Arial", 11, "
446             bold"), fg="blue").grid(row=row, column=1, padx=10, pady=5)
447         row += 1
448
449         frame_btns = tk.Frame(root)
450         frame_btns.grid(row=row, column=0, columnspan=2, pady=15)
451         tk.Button(frame_btns, text="Actualizar gráficas", command=self.
452             _graficar,
453                 bg="#4CAF50", fg="white", font=("Arial", 11)).pack(
454             side=tk.LEFT, padx=10)
455         tk.Button(frame_btns, text="Generar código Arduino", command=
456             self._generar_codigo,
457                 bg="#2196F3", fg="white", font=("Arial", 11)).pack(
458             side=tk.LEFT, padx=10)
459         tk.Button(frame_btns, text="Cerrar", command=self.root.quit,
460                 bg="#f44336", fg="white", font=("Arial", 11)).pack(
461             side=tk.LEFT, padx=10)
462
463         self.status = tk.Label(root, text="", font=("Arial", 10), fg="
464             green")
465         self.status.grid(row=row+1, column=0, columnspan=2, pady=5)
466
467         def _actualizar_parametros(self, event=None):
468             """Muestra/oculta los campos de parámetros específicos según el
469             tipo."""
470             for w in self.frame_params.winfo_children():

```

```

455         w.destroy()
456
457         tipo = self.tipo.get()
458         if tipo == "THIPWM":
459             tk.Label(self.frame_params, text="Amplitud 3ra armónica
(0-1):").pack(side=tk.LEFT, padx=5)
460             tk.Entry(self.frame_params, textvariable=self.param_a3,
width=8).pack(side=tk.LEFT, padx=5)
461         elif tipo == "MPWM":
462             tk.Label(self.frame_params, text="Factor de forma k (1-10):"
).pack(side=tk.LEFT, padx=5)
463             tk.Entry(self.frame_params, textvariable=self.param_k, width
=8).pack(side=tk.LEFT, padx=5)
464
465     def _obtener_referencia(self):
466         """Construye el diccionario de parámetros y genera la referencia
normalizada."""
467         f_ref = float(self.f_ref.get())
468         mf = int(self.mf.get())
469         tipo = self.tipo.get()
470         params = {}
471         if tipo == "THIPWM":
472             params['a3'] = float(self.param_a3.get())
473         elif tipo == "MPWM":
474             params['k'] = float(self.param_k.get())
475         return generar_referencia(tipo, f_ref, mf, params) # en [-1,1]
476
477     def _graficar(self):
478         """Acción del botón 'Actualizar gráficas'."""
479         try:
480             f_ref = float(self.f_ref.get())
481             ma = float(self.ma.get())
482             mf = int(self.mf.get())
483             dead = float(self.dead.get())
484             tipo = self.tipo.get()
485
486             if not (0 <= ma <= 1):
487                 raise ValueError("ma debe estar en [0,1]")
488             if mf < 1:
489                 mf = 1
490                 self.mf.set(str(mf))
491             if dead < 0:
492                 raise ValueError("Tiempo muerto no puede ser negativo")
493
494             ref_norm = self._obtener_referencia()
495             params = {}
496             if tipo == "THIPWM":
497                 params['a3'] = float(self.param_a3.get())
498             elif tipo == "MPWM":
499                 params['k'] = float(self.param_k.get())
500
501             V_rms, thd = graficar_simulacion(f_ref, ma, mf, dead,
ref_norm, tipo, params)
502             self.rms_label.set(f"RMS = {V_rms:.4f}")
503             self.thd_label.set(f"THD = {thd:.2f}%")
504             self.status.config(text="Gráficas actualizadas", fg="green")
505
506         except Exception as e:

```

```

507         messagebox.showerror("Error", str(e))
508         self.status.config(text="Error en gráficas", fg="red")
509
510     def _generar_codigo(self):
511         """Acción del botón 'Generar código Arduino'."""
512         try:
513             f_ref = float(self.f_ref.get())
514             ma = float(self.ma.get())
515             mf = int(self.mf.get())
516             dead = float(self.dead.get())
517             tipo = self.tipo.get()
518
519             if not (0 <= ma <= 1):
520                 raise ValueError("ma debe estar en [0,1]")
521             if mf < 1:
522                 mf = 1
523                 self.mf.set(str(mf))
524             if dead < 0:
525                 raise ValueError("Tiempo muerto no puede ser negativo")
526
527             ref_norm = self._obtener_referencia()
528
529             # ---- Calcular RMS y THD para incluirlos en el encabezado
530             ----
531             t, _, _, diff = simular_senales(f_ref, ma, mf, dead,
532             ref_norm, pts_por_periodo=200)
533             V_rms, _, thd = calcular_rms_thd(t, diff, f_ref)
534
535             # ---- Calcular parámetros del timer (Phase Correct) ----
536             prescaler, ICR, f_c_actual, dead_ticks, OCR1A, OCR1B, N = \
537                 calcular_parametros_timer(f_ref, mf, dead, ma, ref_norm)
538
539             # ---- Parámetros adicionales para el comentario ----
540             params = {}
541             if tipo == "THIPWM":
542                 params['a3'] = float(self.param_a3.get())
543             elif tipo == "MPWM":
544                 params['k'] = float(self.param_k.get())
545
546             # ---- Generar archivo .ino ----
547             generar_archivo_ino(f_ref, ma, mf, dead, tipo, params,
548             prescaler, ICR, f_c_actual, dead_ticks,
549             OCR1A, OCR1B, N, rms=V_rms, thd=thd)
550
551             self.status.config(text="¡Código Arduino generado!", fg="
552             green")
553             messagebox.showinfo("Éxito", "Archivo SPWM.ino generado en
554             la raíz.")
555
556         except Exception as e:
557             messagebox.showerror("Error", str(e))
558             self.status.config(text="Error al generar código", fg="red")
559
560     #
561     =====
562
563     # PUNTO DE ENTRADA

```

```
559 #  
=====
```

```
560  
561 if __name__ == "__main__":  
562     root = tk.Tk()  
563     app = SPWMAApp(root)  
564     root.mainloop()
```

Apéndice C

Anexos: Codigos de Practica Trifasico

C.1. Inversor Trifasico

Este programa controla el inversor trifásico

```
1 /*
2  * Inversor trifásico con dos metodologías de modulación.
3  *
4  * Metodología 1: Modulación por vector espacial (SVPWM) basada en seno/
5  * coseno.
6  * Metodología 2: Conmutación secuencial de seis pasos (tren de pulsos
7  * fijo).
8  *
9  * Permite seleccionar el método y ajustar la frecuencia fundamental (Hz
10 * )
11 * a través del monitor serie (9600 baudios).
12 *
13 * Pines de salida:
14 *   D9  -> PB1 (fase A)
15 *   D10 -> PB2 (fase B)
16 *   D11 -> PB3 (fase C)
17 *
18 * Comandos por serial:
19 *   M1  -> seleccionar metodología 1 (SVPWM)
20 *   M2  -> seleccionar metodología 2 (seis pasos)
21 *   Fxx -> ajustar frecuencia a xx Hz (ej. F50 para 50 Hz)
22 */
23
24 #include <math.h>    // para cos() y sin()
25
26 // ----- Definición de pines y máscaras -----
27 #define PIN_PB1 PB1    // D9
28 #define PIN_PB2 PB2    // D10
29 #define PIN_PB3 PB3    // D11
30
31 #define MASK_PB1 (1 << PIN_PB1)
32 #define MASK_PB2 (1 << PIN_PB2)
33 #define MASK_PB3 (1 << PIN_PB3)
34 #define ALL_MASK (MASK_PB1 | MASK_PB2 | MASK_PB3)
35
36 // Tabla de estados para los 6 sectores del inversor.
37 // Cada entrada contiene los bits que deben estar en HIGH para ese
38 // sector.
```

```

35 const uint8_t stateMasks[6] = {
36     MASK_PB1,                // Sector 0: (1,0,0)
37     MASK_PB1 | MASK_PB2,     // Sector 1: (1,1,0)
38     MASK_PB2,                // Sector 2: (0,1,0)
39     MASK_PB2 | MASK_PB3,     // Sector 3: (0,1,1)
40     MASK_PB3,                // Sector 4: (0,0,1)
41     MASK_PB1 | MASK_PB3      // Sector 5: (1,0,1)
42 };
43
44 // ----- Variables globales -----
45 uint8_t metodo = 1;          // 1 = SVPWM, 2 = seis pasos
46 float frecuencia = 60.0;     // frecuencia en Hz (valor por defecto 60
    Hz)
47
48 // Variables para el método 1 (SVPWM)
49 const unsigned long sampleRate_ms = 1;    // periodo de muestreo en ms
50 unsigned long lastSampleTime = 0;
51 float phase = 0.0;                    // fase acumulada
52
53 // Variables para el método 2 (seis pasos)
54 unsigned long stepDuration_us;          // duración de cada paso en
    microsegundos
55 uint8_t stepIndex = 0;                  // sector actual (0..5)
56 unsigned long lastStepTime_us = 0;
57
58 // ----- Prototipos de funciones -----
59 void aplicarEstado(uint8_t sector);
60 void procesarComandoSerial();
61 void actualizarMetodo1();
62 void actualizarMetodo2();
63
64 // ----- Configuración inicial -----
65 void setup() {
66     // Establecer pines D9, D10, D11 como salida
67     DDRB |= ALL_MASK;
68
69     // Inicializar comunicación serie
70     Serial.begin(9600);
71     Serial.println(F("Sistema de inversor trifásico iniciado."));
72     Serial.println(F("Comandos: M1 (SVPWM), M2 (seis pasos), F<frecuencia>
        (ej. F50)"));
73
74     // Calcular duración de paso para la frecuencia por defecto (60 Hz)
75     actualizarDuracionPaso();
76 }
77
78 // ----- Bucle principal -----
79 void loop() {
80     // Atender comandos del monitor serie
81     procesarComandoSerial();
82
83     // Ejecutar el método seleccionado
84     if (metodo == 1) {
85         actualizarMetodo1();
86     } else {
87         actualizarMetodo2();
88     }
89 }

```



```

90
91 // ----- Aplicar estado (sector) a los pines de salida -----
92 void aplicarEstado(uint8_t sector) {
93     // sector debe estar entre 0 y 5
94     if (sector > 5) sector = 0;
95     // Limpiar los bits de los tres pines y luego poner los
96     // correspondientes al sector
97     PORTB = (PORTB & ~ALL_MASK) | stateMasks[sector];
98 }
99
100 // ----- Procesar comandos recibidos por serial -----
101 void procesarComandoSerial() {
102     if (Serial.available() > 0) {
103         char comando = Serial.read();
104         // Leer el resto del mensaje (hasta nueva línea o fin)
105         String parametro = Serial.readStringUntil('\n');
106         parametro.trim(); // eliminar espacios y saltos de línea
107
108         if (comando == 'M' || comando == 'm') {
109             // Cambio de metodología: se espera '1' o '2'
110             if (parametro.length() > 0) {
111                 char opcion = parametro.charAt(0);
112                 if (opcion == '1') {
113                     metodo = 1;
114                     Serial.println(F("Metodología cambiada a SVPWM (vector
115 espacial)."));
116                 } else if (opcion == '2') {
117                     metodo = 2;
118                     // Reiniciar el contador de pasos para que comience desde el
119                     // primer sector
120                     stepIndex = 0;
121                     lastStepTime_us = micros();
122                     Serial.println(F("Metodología cambiada a seis pasos fijos."));
123                 } else {
124                     Serial.println(F("Opción inválida. Use M1 o M2."));
125                 }
126             }
127         }
128
129         else if (comando == 'F' || comando == 'f') {
130             // Cambio de frecuencia: se espera un número en Hz
131             float nuevaFrec = parametro.toFloat();
132             if (nuevaFrec > 0) {
133                 frecuencia = nuevaFrec;
134                 actualizarDuracionPaso();
135                 Serial.print(F("Frecuencia ajustada a "));
136                 Serial.print(frecuencia);
137                 Serial.println(F(" Hz"));
138             } else {
139                 Serial.println(F("Frecuencia inválida. Ingrese un número
140 positivo."));
141             }
142         }
143
144         else {
145             Serial.println(F("Comando no reconocido. Use M1, M2 o F<frecuencia
146 >."));
147         }
148
149         // Vaciar el buffer por si quedan datos pendientes
150         while (Serial.available()) Serial.read();
151     }
152 }

```

```

143 }
144 }
145
146 // ----- Actualizar parámetros dependientes de la frecuencia -----
147 void actualizarDuracionPaso() {
148     // Para el método 2: duración de cada paso = periodo / 6
149     // periodo (ms) = 1000 / frecuencia
150     // paso (us) = (periodo_ms * 1000) / 6 = (1000/frecuencia * 1000)/6 =
151     // 1e6 / (6 * frecuencia)
152     stepDuration_us = (unsigned long)(1000000.0 / (6.0 * frecuencia));
153     // Para el método 1: no se necesita precalcular, se usa directamente
154     // en el bucle
155 }
156
157 // ----- Actualización del método 1 (SVPWM) -----
158 void actualizarMetodo1() {
159     unsigned long ahora = millis();
160     if (ahora - lastSampleTime >= sampleRate_ms) {
161         lastSampleTime = ahora;
162
163         // Calcular las componentes seno y coseno de la fase actual
164         float signalValue = cos(phase); // componente en fase (Vd)
165         float signalValue2 = sin(phase); // componente en cuadratura (Vq)
166     )
167
168     // Incremento de fase para la frecuencia deseada:
169     // phase += 2*PI * sampleRate_ms / periodo_ms
170     // periodo_ms = 1000 / frecuencia
171     // => phase += 2*PI * 1 / (1000/f) = 2*PI * f / 1000
172     phase += 2.0 * PI * frecuencia / 1000.0;
173     // Mantener fase en rango [0, 2*PI) para evitar desbordamiento
174     if (phase >= 2.0 * PI) phase -= 2.0 * PI;
175
176     // Cálculo del sector según la modulación por vector espacial
177     float fx = signalValue;
178     float fy = signalValue2 / sqrt(3.0);
179     // Determinar N (0..5) según la región del hexágono
180     int N = 2.5 - (fy / abs(fy)) * ((fx > fy) + (fx > -fy) + 0.5);
181     // Asegurar rango
182     if (N < 0) N = 0;
183     if (N > 5) N = 5;
184
185     // Aplicar el estado correspondiente
186     aplicarEstado((uint8_t)N);
187
188     // (Opcional) Enviar datos al plotter serie si se desea
189     // Serial.print(signalValue); Serial.print(",");
190     // Serial.print(signalValue2); Serial.print(",");
191     // Serial.println(N);
192 }
193 }
194
195 // ----- Actualización del método 2 (seis pasos fijos) -----
196 void actualizarMetodo2() {
197     unsigned long ahora_us = micros();
198     // Verificar si ha pasado el tiempo para cambiar al siguiente paso
199     if (ahora_us - lastStepTime_us >= stepDuration_us) {
200         lastStepTime_us = ahora_us;

```

```

198
199     // Avanzar al siguiente sector (0..5)
200     stepIndex++;
201     if (stepIndex >= 6) stepIndex = 0;
202
203     // Aplicar el estado
204     aplicarEstado(stepIndex);
205 }
206 }

```

Este programa es la interfaz grafica del anetrior

```

1  """
2  Interfaz gráfica para controlar el inversor trifásico Arduino.
3  Permite seleccionar puerto COM, metodología (SVPWM o seis pasos)
4  y frecuencia fundamental (Hz). Los comandos se envían por serial
5  a 9600 baudios.
6  """
7
8  import tkinter as tk
9  from tkinter import ttk, messagebox
10 import serial
11 import serial.tools.list_ports
12 import threading
13 import time
14
15 # ----- Clase principal de la GUI -----
16 class InversorGUI:
17     def __init__(self, root):
18         self.root = root
19         self.root.title("Controlador de Inversor Trifásico")
20         self.root.geometry("500x350")
21         self.root.resizable(False, False)
22
23         # Variables de estado
24         self.serial_port = None
25         self.connected = False
26         self.reading_thread = None
27         self.running = False
28
29         # Crear los widgets
30         self.crear_widgets()
31
32         # Actualizar la lista de puertos al inicio
33         self.actualizar_puertos()
34
35     def crear_widgets(self):
36         # ----- Frame de conexión -----
37         frame_conexion = ttk.LabelFrame(self.root, text="Conexión",
padding=10)
38         frame_conexion.pack(fill="x", padx=10, pady=5)
39
40         ttk.Label(frame_conexion, text="Puerto:").grid(row=0, column=0,
padx=5, pady=5, sticky="w")
41         self.combo_puertos = ttk.Combobox(frame_conexion, state="
readonly", width=15)
42         self.combo_puertos.grid(row=0, column=1, padx=5, pady=5)
43
44         self.btn_refrescar = ttk.Button(frame_conexion, text="Refrescar"

```

```

, command=self.actualizar_puertos)
self.btn_refrescar.grid(row=0, column=2, padx=5, pady=5)

self.btn_conectar = ttk.Button(frame_conexion, text="Conectar",
command=self.toggle_conexion)
self.btn_conectar.grid(row=0, column=3, padx=5, pady=5)

self.estado_label = ttk.Label(frame_conexion, text="Estado:
Desconectado", foreground="red")
self.estado_label.grid(row=0, column=4, padx=10, pady=5)

# ----- Frame de configuración (metodología y frecuencia) -----
frame_config = ttk.LabelFrame(self.root, text="Configuración",
padding=10)
frame_config.pack(fill="x", padx=10, pady=5)

# Metodología
ttk.Label(frame_config, text="Metodología:").grid(row=0, column
=0, padx=5, pady=5, sticky="w")
self.metodo_var = tk.StringVar(value="1")
ttk.Radiobutton(frame_config, text="SVPWM (M1)", variable=self.
metodo_var, value="1").grid(row=0, column=1, padx=5, pady=5, sticky="
w")
ttk.Radiobutton(frame_config, text="Seis pasos (M2)", variable=
self.metodo_var, value="2").grid(row=0, column=2, padx=5, pady=5,
sticky="w")

# Frecuencia
ttk.Label(frame_config, text="Frecuencia (Hz):").grid(row=1,
column=0, padx=5, pady=5, sticky="w")
self.frec_var = tk.StringVar(value="60")
self.spin_frec = ttk.Spinbox(frame_config, from_=1, to=200,
increment=1, textvariable=self.frec_var, width=10)
self.spin_frec.grid(row=1, column=1, padx=5, pady=5, sticky="w")

# Botón aplicar
self.btn_aplicar = ttk.Button(frame_config, text="Aplicar
configuración", command=self.enviar_configuracion, state="disabled")
self.btn_aplicar.grid(row=1, column=2, padx=10, pady=5)

# ----- Área de log / mensajes -----
frame_log = ttk.LabelFrame(self.root, text="Mensajes del Arduino
", padding=10)
frame_log.pack(fill="both", expand=True, padx=10, pady=5)

self.text_log = tk.Text(frame_log, height=8, state="disabled",
wrap="word")
self.text_log.pack(fill="both", expand=True)

# Scrollbar para el log
scroll = ttk.Scrollbar(self.text_log, orient="vertical", command
=self.text_log.yview)
scroll.pack(side="right", fill="y")
self.text_log.configure(yscrollcommand=scroll.set)

# ----- Configurar el cierre de la ventana -----
self.root.protocol("WM_DELETE_WINDOW", self.cerrar_aplicacion)

```

```

88 # ----- Funciones de conexión -----
89 def actualizar_puertos(self):
90     """Lista los puertos COM disponibles y los muestra en el
91     combobox."""
92     puertos = [port.device for port in serial.tools.list_ports.
93     comports()]
94     if puertos:
95         self.combo_puertos['values'] = puertos
96         if self.combo_puertos.get() == "":
97             self.combo_puertos.current(0)
98     else:
99         self.combo_puertos['values'] = []
100         self.combo_puertos.set("")
101         if not self.connected:
102             messagebox.showwarning("Sin puertos", "No se detectaron
103             puertos COM disponibles.")
104
105 def toggle_conexion(self):
106     """Conecta o desconecta del puerto serie."""
107     if not self.connected:
108         self.conectar()
109     else:
110         self.desconectar()
111
112 def conectar(self):
113     """Intenta abrir el puerto serie."""
114     puerto = self.combo_puertos.get()
115     if not puerto:
116         messagebox.showerror("Error", "Selecciona un puerto primero.
117 ")
118     return
119
120 try:
121     self.serial_port = serial.Serial(puerto, 9600, timeout=0.1)
122     # Esperar un poco para que el Arduino se reinicie si es
123     necesario
124     time.sleep(2)
125     self.connected = True
126     self.estado_label.config(text="Estado: Conectado",
127 foreground="green")
128     self.btn_conectar.config(text="Desconectar")
129     self.btn_aplicar.config(state="normal")
130     self.combo_puertos.config(state="disabled")
131     self.btn_refrescar.config(state="disabled")
132     self.log_message(f"Conectado a {puerto}")
133
134     # Iniciar hilo de lectura
135     self.running = True
136     self.reading_thread = threading.Thread(target=self.
137 leer_serial, daemon=True)
138     self.reading_thread.start()
139
140 except serial.SerialException as e:
141     messagebox.showerror("Error de conexión", f"No se pudo
142 conectar a {puerto}.\n{e}")
143     self.connected = False
144     self.estado_label.config(text="Estado: Desconectado",
145 foreground="red")

```

```

137         self.btn_conectar.config(text="Conectar")
138         self.btn_aplicar.config(state="disabled")
139
140     def desconectar(self):
141         """Cierra la conexión serie."""
142         self.running = False
143         if self.serial_port and self.serial_port.is_open:
144             self.serial_port.close()
145         self.connected = False
146         self.estado_label.config(text="Estado: Desconectado", foreground
147 = "red")
148         self.btn_conectar.config(text="Conectar")
149         self.btn_aplicar.config(state="disabled")
150         self.combo_puertos.config(state="readonly")
151         self.btn_refrescar.config(state="normal")
152         self.log_message("Desconectado")
153
154     def leer_serial(self):
155         """Hilo que lee continuamente el puerto y muestra los mensajes.
156         """
157         while self.running:
158             try:
159                 if self.serial_port and self.serial_port.is_open:
160                     if self.serial_port.in_waiting > 0:
161                         linea = self.serial_port.readline().decode('utf
162 -8', errors='ignore').strip()
163                         if linea:
164                             self.log_message(f"Arduino: {linea}")
165                             time.sleep(0.05)
166             except Exception as e:
167                 # Si ocurre un error, lo mostramos y salimos del hilo
168                 self.log_message(f"Error en lectura: {e}")
169                 break
170
171     # ----- Envío de comandos -----
172     def enviar_configuracion(self):
173         """Envía la metodología y la frecuencia al Arduino."""
174         if not self.connected or not self.serial_port or not self.
175 serial_port.is_open:
176             messagebox.showerror("Error", "No estás conectado al Arduino
177 .")
178         return
179
180     # Obtener valores
181     metodo = self.metodo_var.get()
182     freq_str = self.freq_var.get()
183
184     # Validar frecuencia
185     try:
186         freq = float(freq_str)
187         if freq <= 0:
188             raise ValueError
189     except ValueError:
190         messagebox.showerror("Error", "La frecuencia debe ser un nú
191 mero positivo.")
192         return
193
194     # Construir comandos

```

```

189 comando_metodo = f"M{metodo}\n"    # M1 o M2
190 comando_frec = f"F{frec:.1f}\n"    # Fxx.x
191
192 # Enviar
193 try:
194     self.serial_port.write(comando_metodo.encode())
195     time.sleep(0.1) # Pequeña pausa entre comandos
196     self.serial_port.write(comando_frec.encode())
197     self.log_message(f"Enviado: {comando_metodo.strip()} y {
comando_frec.strip()}")
198 except Exception as e:
199     messagebox.showerror("Error al enviar", f"No se pudo enviar
el comando.\n{e}")
200     self.log_message(f"Error al enviar: {e}")
201
202 # ----- Utilidades -----
203 def log_message(self, msg):
204     """Añade un mensaje al área de log (desde cualquier hilo)."""
205     self.root.after(0, self._log_message, msg)
206
207 def _log_message(self, msg):
208     """Inserta el mensaje en el widget Text (seguro para hilos)."""
209     self.text_log.config(state="normal")
210     self.text_log.insert(tk.END, f"{msg}\n")
211     self.text_log.see(tk.END)
212     self.text_log.config(state="disabled")
213
214 def cerrar_aplicacion(self):
215     """Cierra la aplicación de forma ordenada."""
216     self.running = False
217     if self.serial_port and self.serial_port.is_open:
218         self.serial_port.close()
219     self.root.destroy()
220
221 # ----- Punto de entrada -----
222 if __name__ == "__main__":
223     root = tk.Tk()
224     app = InversorGUI(root)
225     root.mainloop()

```

C.2. Inversor Trifásico Modulación

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3
4 """
5 Generador PWM para inversor trifásico (Arduino Uno)
6 -----
7 Este programa permite:
8 - Seleccionar el tipo de onda de referencia (SPWM, THIPWM, MPWM, Onda
Cuadrada, SVPWM).
9 - Ajustar parámetros como frecuencia, índice de modulación, relación de
frecuencias.
10 - Visualizar en tres gráficas: referencias+portadora, PWM de las tres
fases (con offset), y tensiones de línea (con offset).

```

```

11 - Calcular el valor RMS y la distorsión armónica total (THD) de las
    tensiones de línea.
12 - Generar el código Arduino (.ino) con las tablas de modulación
    precalculadas para el Timer1 (D9, D10) y Timer2 (D11).
13
14 Dependencias: numpy, matplotlib, tkinter
15 Autor: Prof. Alexander Bueno (adaptado para trifásico)
16 Fecha: 08/2026
17 """
18
19 import tkinter as tk
20 from tkinter import ttk, messagebox
21 import numpy as np
22 import matplotlib.pyplot as plt
23 import os
24
25 #
26 # =====
27 # MÓDULO DE GENERACIÓN DE ONDAS DE REFERENCIA (MODULACIONES ESCALARES)
28 # =====
29
30 def evaluar_referencia(tipo, theta, params):
31     """
32     Evalúa la forma de onda de referencia normalizada (sin escalar por
33     ma)
34     en los ángulos theta.
35
36     Parámetros:
37         tipo (str): 'SPWM', 'THIPWM', 'MPWM' o 'Onda Cuadrada'
38         theta (array): ángulos en radianes
39         params (dict): parámetros adicionales según el tipo:
40             - 'THIPWM': {'a3': float} amplitud de la 3ra armónica (0..1)
41             - 'MPWM': {'k': float} factor de forma (1..10)
42
43     Retorna:
44         array: valores de referencia entre -1 y 1 (sin escalar)
45     """
46     if tipo == "SPWM":
47         ref = np.sin(theta)
48     elif tipo == "THIPWM":
49         a3 = params.get('a3', 0.25)
50         raw = np.sin(theta) + a3 * np.sin(3 * theta)
51         # Normalizar para que el pico sea entre 1 y -1
52         max_raw = np.max(np.abs(raw))
53         ref = raw / max_raw if max_raw > 0 else np.zeros_like(theta)
54     elif tipo == "MPWM":
55         k = params.get('k', 3.0)
56         raw = np.tanh(k * np.sin(theta))
57         max_raw = np.max(np.abs(raw))
58         ref = raw / max_raw if max_raw > 0 else np.zeros_like(theta)
59     elif tipo == "Onda Cuadrada":
60         ref = np.sign(np.sin(theta))
61     else:
62         raise ValueError(f"Tipo de onda desconocido: {tipo}")
63     return ref

```



```

62
63 def generar_referencias_trifasicas(tipo, ma, theta, params):
64     """
65     Genera las tres referencias de fase (va, vb, vc) para un conjunto de
66     ángulos.
67     Retorna: (va, vb, vc) arrays, cada uno escalado por ma y recortado a
68     [-1,1].
69     """
70     # Referencias sin escalar
71     ra = evaluar_referencia(tipo, theta, params)
72     rb = evaluar_referencia(tipo, theta - 2*np.pi/3, params)
73     rc = evaluar_referencia(tipo, theta + 2*np.pi/3, params)
74
75     # Escalar por ma y recortar para evitar saturación
76     va = np.clip(ma * ra, -1, 1)
77     vb = np.clip(ma * rb, -1, 1)
78     vc = np.clip(ma * rc, -1, 1)
79     return va, vb, vc
80
81 #
82
83
84 # =====
85
86 # MÓDULO SVPWM
87 #
88
89 # =====
90
91
92 def svpwm_generate(ma, theta):
93     """
94     Genera las tres señales de modulación usando Space Vector PWM.
95     Retorna: (da, db, dc) arrays de ciclo de trabajo (0..1).
96     """
97     # Referencias senoidales
98     va = ma * np.sin(theta)
99     vb = ma * np.sin(theta - 2*np.pi/3)
100     vc = ma * np.sin(theta + 2*np.pi/3)
101
102     # Transformación alfa-beta (Clarke)
103     v_alpha = va
104     v_beta = (vb - vc) / np.sqrt(3)
105
106     # Calcular sector (0..5)
107     sector = np.floor((np.arctan2(v_beta, v_alpha) + np.pi) / (np.pi/3))
108     % 6
109     sector = sector.astype(int)
110
111     # Para cada punto, calcular T1, T2, T0
112     # Usamos la convención: Vref = ma * exp(j*theta) con Vdc=1 (
113     # normalizado)
114     # T1 = (sqrt(3)*ma*Ts) * sin(60 - theta_shift)
115     # T2 = (sqrt(3)*ma*Ts) * sin(theta_shift)
116     # T0 = Ts - T1 - T2
117     # Luego asignar a los vectores activos según el sector.
118
119     # Pre-calcular constantes
120     sqrt3 = np.sqrt(3)
121     Ts = 1.0 # período normalizado

```

```

112
113 # Ángulo dentro del sector (0..60)
114 theta_sec = np.arctan2(v_beta, v_alpha) - sector * np.pi/3 + np.pi/3
115 # Ajustar para que esté en [0, pi/3]
116 theta_sec = np.mod(theta_sec, np.pi/3)
117
118 # Tiempos normalizados (en fracción de Ts)
119 T1 = (sqrt3 * ma * Ts) * np.sin(np.pi/3 - theta_sec)
120 T2 = (sqrt3 * ma * Ts) * np.sin(theta_sec)
121 T0 = Ts - T1 - T2
122 # Recortar por si ma > 1 (sobremodulación)
123 T1 = np.clip(T1, 0, Ts)
124 T2 = np.clip(T2, 0, Ts)
125 T0 = np.clip(T0, 0, Ts)
126
127 # Asignar tiempos a las fases según sector
128 # Sector 0: V1(100), V2(110)
129 # Sector 1: V3(010), V4(011)
130 # Sector 2: V5(001), V6(101)
131 # etc.
132 # Inicializar arrays
133 da = np.zeros_like(theta)
134 db = np.zeros_like(theta)
135 dc = np.zeros_like(theta)
136
137 # Máscaras por sector
138 mask_s0 = (sector == 0)
139 mask_s1 = (sector == 1)
140 mask_s2 = (sector == 2)
141 mask_s3 = (sector == 3)
142 mask_s4 = (sector == 4)
143 mask_s5 = (sector == 5)
144
145 # Sector 0: V1(100) y V2(110)
146 da[mask_s0] = (T1[mask_s0] + T2[mask_s0] + T0[mask_s0])/2) / Ts
147 db[mask_s0] = (T2[mask_s0] + T0[mask_s0])/2) / Ts
148 dc[mask_s0] = (T0[mask_s0])/2) / Ts
149
150 # Sector 1: V3(010) y V4(011)
151 da[mask_s1] = (T0[mask_s1])/2) / Ts
152 db[mask_s1] = (T1[mask_s1] + T2[mask_s1] + T0[mask_s1])/2) / Ts
153 dc[mask_s1] = (T2[mask_s1] + T0[mask_s1])/2) / Ts
154
155 # Sector 2: V5(001) y V6(101)
156 da[mask_s2] = (T2[mask_s2] + T0[mask_s2])/2) / Ts
157 db[mask_s2] = (T0[mask_s2])/2) / Ts
158 dc[mask_s2] = (T1[mask_s2] + T2[mask_s2] + T0[mask_s2])/2) / Ts
159
160 # Sector 3: V4(011) y V5(001)
161 da[mask_s3] = (T0[mask_s3])/2) / Ts
162 db[mask_s3] = (T1[mask_s3] + T0[mask_s3])/2) / Ts
163 dc[mask_s3] = (T1[mask_s3] + T2[mask_s3] + T0[mask_s3])/2) / Ts
164
165 # Sector 4: V6(101) y V1(100)
166 da[mask_s4] = (T1[mask_s4] + T2[mask_s4] + T0[mask_s4])/2) / Ts
167 db[mask_s4] = (T0[mask_s4])/2) / Ts
168 dc[mask_s4] = (T2[mask_s4] + T0[mask_s4])/2) / Ts
169

```

```

170     # Sector 5: V2(110) y V3(010)
171     da[mask_s5] = (T2[mask_s5] + T0[mask_s5]/2) / Ts
172     db[mask_s5] = (T1[mask_s5] + T2[mask_s5] + T0[mask_s5]/2) / Ts
173     dc[mask_s5] = (T0[mask_s5]/2) / Ts
174
175     # Recortar por si acaso
176     da = np.clip(da, 0, 1)
177     db = np.clip(db, 0, 1)
178     dc = np.clip(dc, 0, 1)
179
180     return da, db, dc
181
182
183 def generar_tablas(tipo, ma, mf, params):
184     """
185     Genera las tablas de ciclo de trabajo (0..1) para un periodo
186     completo.
187     Retorna: (da, db, dc) arrays de tamaño mf.
188     """
189     N = int(mf)
190     theta = np.linspace(0, 2*np.pi, N, endpoint=False)
191
192     if tipo == "SVPWM":
193         da, db, dc = svpwm_generate(ma, theta)
194     else:
195         va, vb, vc = generar_referencias_trifasicas(tipo, ma, theta,
196         params)
197         da = (1 + va) / 2
198         db = (1 + vb) / 2
199         dc = (1 + vc) / 2
200
201     # Asegurar que estén en [0,1]
202     da = np.clip(da, 0, 1)
203     db = np.clip(db, 0, 1)
204     dc = np.clip(dc, 0, 1)
205
206     return da, db, dc
207
208 #
209 # =====
210
211 # MÓDULO DE CÁLCULO DEL TEMPORIZADOR Y GENERACIÓN DEL CÓDIGO ARDUINO
212 #
213 # =====
214
215 def calcular_parametros_timer(f_ref, mf):
216     """
217     Calcula la configuración óptima para Timer1 y Timer2 para un Arduino
218     Uno (16 MHz)
219     de modo que ambos timers operen a la misma frecuencia de portadora
220     f_c = mf * f_ref.
221
222     Timer1: Fast PWM mode 14 (TOP=ICR1), salidas OC1A (D9) y OC1B (D10)
223     Timer2: Fast PWM mode 3 (TOP=0xFF), salida OC2A (D11)
224
225     Retorna: (prescaler, ICR1, f_c_actual, N)

```

```

220 """
221 F_CLK = 16000000
222 PRESCALER_OPTS = [1, 8, 64, 256, 1024]
223 f_c = mf * f_ref
224
225 # Para Timer2: f_c = F_CLK / (prescaler * 256)
226 # Buscamos el prescaler que dé una frecuencia lo más cercana a f_c
227 best_p = 1
228 best_error = float('inf')
229 for p in PRESCALER_OPTS:
230     f_c_actual = F_CLK / (p * 256)
231     error = abs(f_c_actual - f_c)
232     if error < best_error:
233         best_error = error
234         best_p = p
235
236 # Con ese prescaler, calculamos ICR1 para Timer1: f_c = F_CLK / (p *
237 # (ICR1+1))
238 # Queremos que Timer1 tenga la misma frecuencia que Timer2
239 ICR1 = int(round(F_CLK / (best_p * f_c) - 1))
240 if ICR1 < 1:
241     ICR1 = 1
242 elif ICR1 > 65535:
243     ICR1 = 65535
244
245 # Recalcular frecuencia real de Timer1
246 f_c_actual = F_CLK / (best_p * (ICR1 + 1))
247
248 return best_p, ICR1, f_c_actual
249
250 def generar_archivo_ino(f_ref, ma, mf, tipo, params,
251                        prescaler, ICR1, f_c_actual,
252                        da, db, dc, rms_ab=0.0, thd_ab=0.0):
253     """
254     Escribe el archivo .ino en la raíz.
255     Configura Timer1 y Timer2 en Fast PWM.
256     """
257     N = len(da)
258
259     # Escalar a valores para OCR
260     # Timer1: OCR1A = duty * ICR1, OCR1B = duty * ICR1
261     OCR1A = np.round(da * ICR1).astype(int)
262     OCR1B = np.round(db * ICR1).astype(int)
263     # Timer2: OCR2A = duty * 255
264     OCR2A = np.round(dc * 255).astype(int)
265
266     # Asegurar límites
267     OCR1A = np.clip(OCR1A, 0, ICR1)
268     OCR1B = np.clip(OCR1B, 0, ICR1)
269     OCR2A = np.clip(OCR2A, 0, 255)
270
271     ruta = "SPWM_3F.ino"
272
273     with open(ruta, "w") as f:
274         f.write("//
=====\\n")
275         f.write("// PWM para inversor trifásico (Arduino Uno)\\n")

```

```

276     f.write("//  Generado automáticamente por Python\n")
277     f.write("//  Modo: Fast PWM, Timer1 (D9, D10) y Timer2 (D11)\n")
278     f.write("//
=====
279     f.write(f"//  f_ref    = {f_ref:.2f} Hz\n")
280     f.write(f"//  ma      = {ma:.3f}\n")
281     f.write(f"//  mf      = {mf}\n")
282     f.write(f"//  Tipo    = {tipo}\n")
283     if params:
284         f.write(f"//  Params  = {params}\n")
285     f.write(f"//  f_c      = {f_c_actual:.2f} Hz (real)\n")
286     f.write(f"//  Prescaler = {prescaler}, ICR1 = {ICR1}\n")
287     f.write(f"//  Muestras = {N}\n")
288     f.write(f"//  RMS (Vab) = {rms_ab:.4f} (tensión de línea
normalizada)\n")
289     f.write(f"//  THD (Vab) = {thd_ab:.2f}%\n")
290     f.write("//
=====
)

291
292     f.write("#include <avr/pgmspace.h>\n\n")
293
294     f.write("void setup() {\n")
295     f.write("  // ---- Configuración de pines como salidas (ya lo
hacen los timers) ----\n")
296     f.write("  // D9 (PB1), D10 (PB2), D11 (PB3)\n")
297     f.write("  DDRB |= (1 << PB1) | (1 << PB2) | (1 << PB3);\n\n")
298
299     f.write("  // ---- Timer1: Fast PWM, TOP = ICR1 (modo 14) ----\n"
")
300     f.write("  TCCR1A = 0;\n")
301     f.write("  TCCR1B = 0;\n")
302     f.write("  // WGM13=1, WGM12=0, WGM11=1, WGM10=0 -> modo 14\n")
303     f.write("  TCCR1A |= (1 << WGM11);\n")
304     f.write("  TCCR1B |= (1 << WGM13);\n")
305     f.write("  TCCR1B &= ~(1 << WGM12);\n")
306     f.write("  TCCR1A &= ~(1 << WGM10);\n\n")
307
308     f.write("  // OC1A no inversor (COM1A1=1, COM1A0=0)\n")
309     f.write("  TCCR1A |= (1 << COM1A1);\n")
310     f.write("  TCCR1A &= ~(1 << COM1A0);\n")
311     f.write("  // OC1B no inversor (COM1B1=1, COM1B0=0)\n")
312     f.write("  TCCR1A |= (1 << COM1B1);\n")
313     f.write("  TCCR1A &= ~(1 << COM1B0);\n\n")
314
315     f.write(f"  ICR1 = {ICR1};\n")
316     f.write(f"  OCR1A = {OCR1A[0]};\n")
317     f.write(f"  OCR1B = {OCR1B[0]};\n\n")
318
319     f.write("  // ---- Timer2: Fast PWM, TOP = 0xFF (modo 3) ----\n"
)
320     f.write("  TCCR2A = 0;\n")
321     f.write("  TCCR2B = 0;\n")
322     f.write("  // WGM22=1, WGM21=0, WGM20=1 -> modo 3 (Fast PWM, TOP
=0xFF)\n")
323     f.write("  TCCR2A |= (1 << WGM22) | (1 << WGM20);\n")
324     f.write("  TCCR2A &= ~(1 << WGM21);\n")
325     f.write("  // OC2A no inversor (COM2A1=1, COM2A0=0)\n")

```

```

326     f.write("    TCCR2A |= (1 << COM2A1);\n")
327     f.write("    TCCR2A &= ~(1 << COM2A0);\n")
328     f.write(f"    OCR2A = {OCR2A[0]};\n\n")
329
330     f.write("    // ---- Prescaler ----\n")
331     if prescaler == 1:
332         f.write("    TCCR1B |= (1 << CS10);\n")
333         f.write("    TCCR2B |= (1 << CS20);\n")
334     elif prescaler == 8:
335         f.write("    TCCR1B |= (1 << CS11);\n")
336         f.write("    TCCR2B |= (1 << CS21);\n")
337     elif prescaler == 64:
338         f.write("    TCCR1B |= (1 << CS11) | (1 << CS10);\n")
339         f.write("    TCCR2B |= (1 << CS22);\n")
340     elif prescaler == 256:
341         f.write("    TCCR1B |= (1 << CS12);\n")
342         f.write("    TCCR2B |= (1 << CS22) | (1 << CS21);\n")
343     elif prescaler == 1024:
344         f.write("    TCCR1B |= (1 << CS12) | (1 << CS10);\n")
345         f.write("    TCCR2B |= (1 << CS22) | (1 << CS21) | (1 << CS20)
;\n")
346     f.write("\n")
347
348     f.write("    // Habilitar interrupción por desbordamiento de
Timer1\n")
349     f.write("    TIMSK1 |= (1 << TOIE1);\n")
350     f.write("}\n\n")
351
352     f.write(f"const int N = {N};\n\n")
353
354     f.write("const unsigned int OCR1A_table[N] PROGMEM = {\n")
355     for i, v in enumerate(OCR1A):
356         f.write(f"    {v}{',' if i < N-1 else ''}\n")
357     f.write("};\n\n")
358
359     f.write("const unsigned int OCR1B_table[N] PROGMEM = {\n")
360     for i, v in enumerate(OCR1B):
361         f.write(f"    {v}{',' if i < N-1 else ''}\n")
362     f.write("};\n\n")
363
364     f.write("const unsigned char OCR2A_table[N] PROGMEM = {\n")
365     for i, v in enumerate(OCR2A):
366         f.write(f"    {v}{',' if i < N-1 else ''}\n")
367     f.write("};\n\n")
368
369     f.write("volatile unsigned char index = 0;\n\n")
370
371     f.write("ISR(TIMER1_OVF_vect) {\n")
372     f.write("    index++;\n")
373     f.write("    if (index >= N) index = 0;\n")
374     f.write("    OCR1A = pgm_read_word(&OCR1A_table[index]);\n")
375     f.write("    OCR1B = pgm_read_word(&OCR1B_table[index]);\n")
376     f.write("    OCR2A = pgm_read_byte(&OCR2A_table[index]);\n")
377     f.write("}\n\n")
378
379     f.write("void loop() {\n")
380     f.write("    // Todo el control se realiza por interrupción\n")
381     f.write("}\n")

```

```

382
383     print(f"Archivo generado: {ruta}")
384
385
386 #
=====
387 # MÓDULO DE SIMULACIÓN, CÁLCULO DE RMS Y THD, Y GRÁFICAS
388 #
=====

389
390 def simular_senales(f_ref, ma, mf, ref_a, ref_b, ref_c, pts_por_periodo
=200):
391     """
392     Simula las señales PWM para las tres fases y las tensiones de línea.
393     Utiliza portadora triangular en [-1, 1] y comparación directa con la
394     referencia.
395     Retorna: t, outA, outB, outC, Vab, Vbc, Vca
396     """
397     f_c = mf * f_ref
398     T_c = 1.0 / f_c
399     num_periodos = 2 * mf
400     tiempo_total = num_periodos * T_c
401     n_pts = num_periodos * pts_por_periodo
402     t = np.linspace(0, tiempo_total, n_pts, endpoint=False)
403
404     # Portadora triangular en [-1, 1]
405     fase = (t % T_c) / T_c
406     portadora = 1 - 4 * np.abs(fase - 0.5)
407
408     # Referencias repetidas para dos ciclos y expandidas
409     ref_a_repetida = np.repeat(np.tile(ref_a, 2), pts_por_periodo)
410     ref_b_repetida = np.repeat(np.tile(ref_b, 2), pts_por_periodo)
411     ref_c_repetida = np.repeat(np.tile(ref_c, 2), pts_por_periodo)
412
413     # PWM: 1 si referencia > portadora
414     outA = (ref_a_repetida > portadora).astype(float)
415     outB = (ref_b_repetida > portadora).astype(float)
416     outC = (ref_c_repetida > portadora).astype(float)
417
418     # Tensiones de línea (normalizadas)
419     Vab = outA - outB
420     Vbc = outB - outC
421     Vca = outC - outA
422
423     return t, outA, outB, outC, Vab, Vbc, Vca
424
425 def calcular_rms_thd(t, senal, f_ref):
426     """
427     Calcula el RMS total, el RMS de la fundamental y el THD (%).
428     """
429     dt = t[1] - t[0]
430     N = len(senal)
431     y = senal - np.mean(senal)
432     V_rms = np.sqrt(np.mean(y**2))
433     if V_rms < 1e-12:

```

```

434         return 0.0, 0.0, 0.0
435
436     Y = np.fft.rfft(y)
437     freqs = np.fft.rfftfreq(N, dt)
438     idx_fund = np.argmin(np.abs(freqs - f_ref))
439     if idx_fund == 0:
440         V1_peak = Y[0] / N
441     else:
442         V1_peak = 2 * np.abs(Y[idx_fund]) / N
443     V1_rms = V1_peak / np.sqrt(2)
444
445     max_armonica = 50
446     idx_max = min(len(Y)-1, int(max_armonica * f_ref / (freqs[1] - freqs
447 [0])) + 1)
448     suma_cuadrados = 0.0
449     for k in range(1, idx_max+1):
450         if k == idx_fund:
451             continue
452         if k == 0:
453             Vk_peak = Y[0] / N
454         else:
455             Vk_peak = 2 * np.abs(Y[k]) / N
456         suma_cuadrados += (Vk_peak / np.sqrt(2))**2
457     thd = np.sqrt(suma_cuadrados) / V1_rms if V1_rms > 1e-12 else 0.0
458     return V_rms, V1_rms, thd * 100
459
460 def graficar_simulacion(f_ref, ma, mf, ref_a, ref_b, ref_c, tipo, params
461 ):
462     """
463     Muestra tres subplots en vertical:
464     1) Referencias + portadora triangular
465     2) PWM de las tres fases desplazadas verticalmente (offset
466     aumentado)
467     3) Tensiones de línea (Vab, Vbc, Vca) desplazadas verticalmente
468     para mejor visualización
469     Retorna (RMS_Vab, THD_Vab).
470     """
471     # Simulación
472     t, outA, outB, outC, Vab, Vbc, Vca = simular_senales(f_ref, ma, mf,
473     ref_a, ref_b, ref_c)
474
475     # Calcular RMS y THD para Vab
476     V_rms, _, thd = calcular_rms_thd(t, Vab, f_ref)
477
478     # Referencia y portadora de alta resolución para la gráfica
479     f_c = mf * f_ref
480     T_c = 1.0 / f_c
481     num_periodos = 2 * mf
482     tiempo_total = num_periodos * T_c
483     pts_high = 1000 * num_periodos
484     t_high = np.linspace(0, tiempo_total, pts_high, endpoint=False)
485     theta_high = 2 * np.pi * f_ref * t_high
486
487     # Obtener referencias escaladas para la gráfica
488     va_high, vb_high, vc_high = generar_referencias_trifasicas(tipo, ma,
489     theta_high, params)

```



```

486 # Portadora triangular
487 fase_high = (t_high % T_c) / T_c
488 portadora_high = 1 - 4 * np.abs(fase_high - 0.5)
489
490 # Crear figura con 3 subplots verticales
491 fig, axs = plt.subplots(3, 1, figsize=(14, 12))
492 fig.suptitle(
493     f'PWM Trifásico con portadora triangular - 2 ciclos completos\n'
494     f'f_ref = {f_ref:.1f} Hz, ma = {ma:.2f}, mf = {mf}\n'
495     f'Tipo: {tipo} | RMS(Vab) = {V_rms:.4f} | THD(Vab) = {thd:.2'
496     f}% ',
497     fontsize=14
498 )
499
500 # ---- Subplot 1: Referencias y portadora ----
501 axs[0].plot(t_high, va_high, 'b', lw=1.5, label='Va')
502 axs[0].plot(t_high, vb_high, 'g', lw=1.5, label='Vb')
503 axs[0].plot(t_high, vc_high, 'r', lw=1.5, label='Vc')
504 axs[0].plot(t_high, portadora_high, 'k', lw=0.6, alpha=0.5, label='
Portadora')
505 axs[0].axhline(y=0, color='gray', linestyle='--', linewidth=0.5)
506 axs[0].set_ylabel('Amplitud')
507 axs[0].grid(True)
508 axs[0].legend()
509 axs[0].set_title('Referencias + Portadora (alta resolución)')
510
511 # ---- Subplot 2: PWM de las tres fases (desplazadas con offset
mayor) ----
512 offset_pwm = 1.5 # Aumentado para evitar superposición
513 axs[1].plot(t, outA + 2*offset_pwm, 'b', drawstyle='steps-post', lw
=0.8, label='Fase A (D9)')
514 axs[1].plot(t, outB + offset_pwm, 'g', drawstyle='steps-post', lw
=0.8, label='Fase B (D10)')
515 axs[1].plot(t, outC, 'r', drawstyle='steps-post', lw=0.8, label='
Fase C (D11)')
516 axs[1].axhline(y=2*offset_pwm, color='gray', linestyle=':',
linewidth=0.5)
517 axs[1].axhline(y=offset_pwm, color='gray', linestyle=':', linewidth
=0.5)
518 axs[1].axhline(y=0, color='gray', linestyle=':', linewidth=0.5)
519 axs[1].set_ylabel('Nivel desplazado')
520 axs[1].set_xlabel('Tiempo (s)')
521 axs[1].grid(True)
522 axs[1].legend()
523 axs[1].set_title('Señales PWM de las tres fases (desplazadas
verticalmente)')
524 axs[1].set_xlim(t[0], t[-1])
525 # Ajustar límites y para que quede bien
526 ymin = -0.5
527 ymax = 2*offset_pwm + 1.2
528 axs[1].set_ylim(ymin, ymax)
529
530 # ---- Subplot 3: Tensiones de línea con desplazamiento vertical
----
531 offset_linea = 2.5 # Desplazamiento para separar las tres tensiones
532 axs[2].plot(t, Vab + offset_linea, 'b', drawstyle='steps-post', lw
=0.8, label='Vab + offset')

```

```

533     axs[2].plot(t, Vbc, 'g', drawstyle='steps-post', lw=0.8, label='Vbc'
534 )
535     axs[2].plot(t, Vca - offset_linea, 'r', drawstyle='steps-post', lw
536 =0.8, label='Vca - offset')
537     axs[2].axhline(y=offset_linea, color='gray', linestyle=':',
538 linewidth=0.5)
539     axs[2].axhline(y=0, color='gray', linestyle=':', linewidth=0.5)
540     axs[2].axhline(y=-offset_linea, color='gray', linestyle=':',
541 linewidth=0.5)
542     axs[2].set_ylabel('Tensión desplazada')
543     axs[2].set_xlabel('Tiempo (s)')
544     axs[2].grid(True)
545     axs[2].legend()
546     axs[2].set_title('Tensiones de línea (Vab, Vbc, Vca) desplazadas
547 verticalmente')
548     axs[2].set_xlim(t[0], t[-1])
549     # Ajustar límites
550     ymin = -offset_linea - 1.2
551     ymax = offset_linea + 1.2
552     axs[2].set_ylim(ymin, ymax)
553
554     plt.tight_layout()
555     plt.show()
556     return V_rms, thd
557
558 #
559
560 # INTERFAZ GRÁFICA (Tkinter)
561 #
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
class SPWMTrifasicoApp:
    """Aplicación principal con interfaz gráfica para inversor trifásico
    ."""

    def __init__(self, root):
        self.root = root
        root.title("Generador PWM para Inversor Trifásico (Arduino Uno)")
    )

    root.geometry("620x620")
    root.resizable(False, False)

    # Variables de los parámetros
    self.f_ref = tk.StringVar(value="60.0")
    self.ma = tk.StringVar(value="0.8")
    self.mf = tk.StringVar(value="12")
    self.tipo = tk.StringVar(value="SPWM")
    self.param_a3 = tk.StringVar(value="0.25")
    self.param_k = tk.StringVar(value="3.0")

    # Variables para mostrar resultados
    self.rms_label = tk.StringVar(value="RMS(Vab): ---")
    self.thd_label = tk.StringVar(value="THD(Vab): ---%")

    self._crear_widgets()

```

```

580
581     def _crear_widgets(self):
582         root = self.root
583         row = 0
584
585         tk.Label(root, text="Frecuencia de referencia (Hz):").grid(row=
586 row, column=0, padx=10, pady=5, sticky='e')
587         tk.Entry(root, textvariable=self.f_ref, width=15).grid(row=row,
588 column=1, padx=10, pady=5, sticky='w')
589         row += 1
590
591         tk.Label(root, text="Índice de modulación ma (0-1.15):").grid(
592 row=row, column=0, padx=10, pady=5, sticky='e')
593         tk.Entry(root, textvariable=self.ma, width=15).grid(row=row,
594 column=1, padx=10, pady=5, sticky='w')
595         row += 1
596
597         tk.Label(root, text="Índice de modulación mf (entero):").grid(
598 row=row, column=0, padx=10, pady=5, sticky='e')
599         tk.Entry(root, textvariable=self.mf, width=15).grid(row=row,
600 column=1, padx=10, pady=5, sticky='w')
601         row += 1
602
603         tk.Label(root, text="Tipo de modulación:").grid(row=row, column
604 =0, padx=10, pady=5, sticky='e')
605         self.tipo_combo = ttk.Combobox(root, textvariable=self.tipo,
606 values=["SPWM", "THIPWM", "MPWM",
607 "Onda Cuadrada", "SVPWM"],
608 state="readonly")
609         self.tipo_combo.grid(row=row, column=1, padx=10, pady=5, sticky=
610 'w')
611         self.tipo_combo.bind("<<ComboboxSelected>>", self.
612 _actualizar_parametros)
613         row += 1
614
615         self.frame_params = tk.Frame(root)
616         self.frame_params.grid(row=row, column=0, columnspan=2, pady=5)
617         self._actualizar_parametros()
618         row += 1
619
620         tk.Label(root, textvariable=self.rms_label, font=("Arial", 11, "
621 bold"), fg="blue").grid(row=row, column=0, padx=10, pady=5)
622         tk.Label(root, textvariable=self.thd_label, font=("Arial", 11, "
623 bold"), fg="blue").grid(row=row, column=1, padx=10, pady=5)
624         row += 1
625
626         frame_btns = tk.Frame(root)
627         frame_btns.grid(row=row, column=0, columnspan=2, pady=15)
628         tk.Button(frame_btns, text="Actualizar gráficas", command=self.
629 _graficar,
630 bg="#4CAF50", fg="white", font=("Arial", 11)).pack(
631 side=tk.LEFT, padx=10)
632         tk.Button(frame_btns, text="Generar código Arduino", command=
633 self._generar_codigo,
634 bg="#2196F3", fg="white", font=("Arial", 11)).pack(
635 side=tk.LEFT, padx=10)
636         tk.Button(frame_btns, text="Cerrar", command=self.root.quit,
637 bg="#f44336", fg="white", font=("Arial", 11)).pack(

```

```

side=tk.LEFT, padx=10)
622
623     self.status = tk.Label(root, text="", font=("Arial", 10), fg="
green")
624     self.status.grid(row=row+1, column=0, columnspan=2, pady=5)
625
626     def _actualizar_parametros(self, event=None):
627         for w in self.frame_params.wininfo_children():
628             w.destroy()
629
630         tipo = self.tipo.get()
631         if tipo == "THIPWM":
632             tk.Label(self.frame_params, text="Amplitud 3ra armónica
(0-1):").pack(side=tk.LEFT, padx=5)
633             tk.Entry(self.frame_params, textvariable=self.param_a3,
width=8).pack(side=tk.LEFT, padx=5)
634             elif tipo == "MPWM":
635                 tk.Label(self.frame_params, text="Factor de forma k (1-10):"
).pack(side=tk.LEFT, padx=5)
636                 tk.Entry(self.frame_params, textvariable=self.param_k, width
=8).pack(side=tk.LEFT, padx=5)
637
638         def _obtener_parametros(self):
639             f_ref = float(self.f_ref.get())
640             ma = float(self.ma.get())
641             mf = int(self.mf.get())
642             tipo = self.tipo.get()
643             params = {}
644             if tipo == "THIPWM":
645                 params['a3'] = float(self.param_a3.get())
646             elif tipo == "MPWM":
647                 params['k'] = float(self.param_k.get())
648             return f_ref, ma, mf, tipo, params
649
650         def _graficar(self):
651             try:
652                 f_ref, ma, mf, tipo, params = self._obtener_parametros()
653
654                 if not (0 <= ma <= 1.15):
655                     raise ValueError("ma debe estar en [0, 1.15]")
656                 if mf < 1:
657                     mf = 1
658                     self.mf.set(str(mf))
659
660                 # Generar tablas de referencia
661                 theta = np.linspace(0, 2*np.pi, mf, endpoint=False)
662                 if tipo == "SVPWM":
663                     da, db, dc = svpwm_generate(ma, theta)
664                     # Para la gráfica, necesitamos las referencias
equivalentes
665                     # Podemos usar las referencias senoidales originales
para mostrarlas
666                     va = ma * np.sin(theta)
667                     vb = ma * np.sin(theta - 2*np.pi/3)
668                     vc = ma * np.sin(theta + 2*np.pi/3)
669                     # Pero para la simulación usamos las señales de modulaci
ón reales
670                     ref_a = 2*da - 1

```

```

671         ref_b = 2*db - 1
672         ref_c = 2*dc - 1
673         # Para la gráfica de referencias, usamos las senoidales
para claridad
674         ref_a_plot = va
675         ref_b_plot = vb
676         ref_c_plot = vc
677     else:
678         va, vb, vc = generar_referencias_trifasicas(tipo, ma,
theta, params)
679         ref_a = va
680         ref_b = vb
681         ref_c = vc
682         ref_a_plot = va
683         ref_b_plot = vb
684         ref_c_plot = vc
685
686         # Graficar (pasar las referencias de modulación reales)
687         V_rms, thd = graficar_simulacion(f_ref, ma, mf, ref_a, ref_b
, ref_c, tipo, params)
688         self.rms_label.set(f"RMS(Vab) = {V_rms:.4f}")
689         self.thd_label.set(f"THD(Vab) = {thd:.2f}%")
690         self.status.config(text="Gráficas actualizadas", fg="green")
691
692     except Exception as e:
693         messagebox.showerror("Error", str(e))
694         self.status.config(text="Error en gráficas", fg="red")
695
696     def _generar_codigo(self):
697     try:
698         f_ref, ma, mf, tipo, params = self._obtener_parametros()
699
700         if not (0 <= ma <= 1.15):
701             raise ValueError("ma debe estar en [0, 1.15]")
702         if mf < 1:
703             mf = 1
704             self.mf.set(str(mf))
705
706         # Generar tablas de ciclo de trabajo
707         da, db, dc = generar_tablas(tipo, ma, mf, params)
708
709         # Calcular RMS y THD para Vab (para el encabezado)
710         theta = np.linspace(0, 2*np.pi, mf, endpoint=False)
711         if tipo == "SVPWM":
712             ref_a = 2*da - 1
713             ref_b = 2*db - 1
714             ref_c = 2*dc - 1
715         else:
716             va, vb, vc = generar_referencias_trifasicas(tipo, ma,
theta, params)
717             ref_a = va
718             ref_b = vb
719             ref_c = vc
720
721         t, _, _, _, Vab, _, _ = simular_senales(f_ref, ma, mf, ref_a
, ref_b, ref_c)
722         V_rms, _, thd = calcular_rms_thd(t, Vab, f_ref)
723

```

```

724         # Calcular parámetros del timer
725         prescaler, ICR1, f_c_actual = calcular_parametros_timer(
f_ref, mf)
726
727         # Generar .ino
728         generar_archivo_ino(f_ref, ma, mf, tipo, params,
729                             prescaler, ICR1, f_c_actual,
730                             da, db, dc, rms_ab=V_rms, thd_ab=thd)
731
732         self.status.config(text="¡Código Arduino generado! (
inversor_trifasico.ino)", fg="green")
733         messagebox.showinfo("Éxito", "Archivo inversor_trifasico.ino
generado en la raíz.")
734
735         except Exception as e:
736             messagebox.showerror("Error", str(e))
737             self.status.config(text="Error al generar código", fg="red")
738
739
740 #
=====
741 # PUNTO DE ENTRADA
742 #
=====
743
744 if __name__ == "__main__":
745     root = tk.Tk()
746     app = SPWMTrifasicoApp(root)
747     root.mainloop()

```

Bibliografía

- [1] N. Mohan, T. M. Undeland, and W. P. Robbins, *Power Electronics: Converters, Applications, and Design*, 3rd ed. John Wiley & Sons, 2003.
- [2] M. H. Rashid, *Power Electronics: Circuits, Devices, and Applications*, 4th ed. Pearson, 2014.
- [3] R. W. Erickson and D. Maksimovic, *Fundamentals of Power Electronics*, 2nd ed. Springer, 2001.
- [4] D. G. Holmes and T. A. Lipo, *Pulse Width Modulation for Power Converters: Principles and Practice*. Wiley-IEEE Press, 2003.
- [5] B. K. Bose, *Modern Power Electronics and AC Drives*. Prentice Hall, 2002.
- [6] H. W. Van der Broeck, H. C. Skudelny, and G. V. Stanke, “Analysis and realization of a pulsewidth modulator based on voltage space vectors,” *IEEE Trans. Ind. Appl.*, vol. 24, no. 1, pp. 142–150, Jan./Feb. 1988.
- [7] J. Holtz, “Pulsewidth modulation—A survey,” *IEEE Trans. Ind. Electron.*, vol. 39, no. 5, pp. 410–420, Oct. 1992.
- [8] A. M. Hava, R. J. Kerkman, and T. A. Lipo, “Simple analytical and graphical methods for carrier-based PWM-VSI drives,” *IEEE Trans. Power Electron.*, vol. 14, no. 1, pp. 49–61, Jan. 1999.
- [9] Atmel Corporation, “ATmega328P Datasheet,” Microchip Technology Inc., 2016.